HOSPITAL MANAGEMENT SYSTEM

PROJECT REPORT

Course Name: Web Application Development

Instructor: Dr. Lê Duy Tân

Submission Date: May 31st, 2026

Prepared By:

ID	Name
ITCSIU24029	Trần Đình Hưng
ITCSIU24030	Nguyễn Hưng
ITCSIU24038	Phạm Song Gia Khánh
ITCSIU24042	Nguyễn Thế Khoa

Abstract

This project is a Hospital Management System which is the topic Z of the final assessment for the Web Application Development course. The application demonstrates the operational inefficiencies faced by small to medium hospitals that still rely on manual record keeping and fragmented information management. By digitizing patient records, room assignments, doctor schedules, appointments, treatment tracking, transport logistics, and billing processes, the system reduces administrative errors and improves communication between hospital staff.

The system was built using the Spring Boot framework for the backend, MySQL as the relational database, and a single-page application (SPA) frontend using HTML, CSS, and JavaScript. The application implements six distinct user roles: Administrator, Receptionist, Doctor, Nurse, Ward Boy, and Patient. Each role has role-based access control and customized access to features. Core functionalities include comprehensive CRUD operations for patients, doctors, staff, rooms, treatments, admissions, appointments, and bills. Advanced features include an interactive statistical dashboard, automated bill generation with discount and tax calculation, PDF bill export using iText, and patient transport task management.

Table of Contents

List of Tables

List of Figures

1. Introduction

This report documents our final project for the Web Application Development course, a Hospital Management System named Hospitalz. We spent almost five weeks on this project, from initial planning to deployment, and it ended up being more complex than we expected at first.

The idea came from noticing that smaller hospitals in most areas of Ho Chi Minh city still rely heavily on paper-based record keeping. That creates obvious problems: lost files, manual billing errors, no real-time room availability tracking, and poor communication between departments. Our system digitizes these processes so that hospital staff can manage patients, rooms, appointments, treatments, transport tasks, and billing from a single web interface.

We ended up with six distinct user roles. Admin manages the whole system, including user accounts, room management, staff assignments, and audit statistics. Receptionists handle patient registration, admissions, discharge, appointments, and billing. Doctors manage patient diagnoses, treatment plans, prescriptions, and appointment schedules. Nurses track patient vitals and see which patients they are assigned to. Ward Boys manage room cleaning and patient transport between rooms. Patients can log in to view their own records, current treatment, and billing status. We tried to make the permissions realistic: a nurse cannot access admin pages, and a patient cannot see other patients' data.

Technically, we chose Spring Boot 4.0.6 with Java 21 for the backend because it is recommended by the instructor and more productive than raw servlets.

MySQL was our database choice since we both had used it before in the Principles of Database Management course last semester. The frontend is a single-page application (SPA) built with HTML, CSS, and JavaScript. We did not use Thymeleaf or Bootstrap because we wanted full control over the UI and preferred a modern SPA architecture where the backend only serves JSON data via REST APIs. The frontend consists of auth.html for authentication and index.html for the main dashboard, with JavaScript modules handling all interactivity.

Our three advanced features are: (1) an interactive dashboard with Chart.js showing hospital statistics like admission trends and revenue, (2) automated bill generation that calculates room charges based on stay duration plus treatment, doctor, and outpatient costs, with support for discounts, tax, partial payments, refunds, and voiding, and (3) PDF bill export using iText with professional styling and itemized breakdowns.

Repository:

https://github.com/Noobovich1/CodeStrike-Hospital_Management

2. Requirements Analysis

2.1 Problem Statement

Hospitals, especially the smaller and medium-sized ones, still manage a surprising amount of data on paper.

Firstly, patient records are physical files. They get lost, damaged, or misfiled. When a patient returns for follow-up, staff sometimes can not find their history, which is dangerous if the patient has allergies or chronic conditions. Secondly, room management is chaotic. Reception staff literally walk around to check if rooms are free, or they maintain a whiteboard that someone forgets to update. This leads to dual bookings or patients waiting in hallways because the system says a room is free when it is actually occupied.Thirdly, billing is done by hand. Someone counts the days of stay, looks up the room rate, adds doctor fees, and calculates the total on a calculator. Mistakes can happen because we have heard of patients being overcharged because the discharge date was counted wrong, or undercharged because a treatment was missed. Lastly, communication between staff is separated. A doctor might update a treatment plan, but the nurse does not know until the doctor physically tells them or writes it on a paper chart. There is no central place where everyone sees the current status.

Our system solves this by keeping everything digital and centralized. Patient records are searchable instantly. Room status updates automatically when someone is admitted or discharged. Bills calculate themselves from the actual data. And role-based access means each staff member only sees what they need, which keeps things organized.

2.2 User Stories

During Week 1, we wrote around 20 user stories to figure out what the system actually needed to do. We organized them by role and marked priorities so we knew what to build first if we ran out of time.

- **Admin stories** were mostly about control and oversight. The admin needs to create staff and receptionist accounts, deactivate them when someone leaves, manage rooms, view statistics, and check who changed what in the system. These were all "Must Have" because without admin control, the system cannot be managed.

- **Receptionist stories** focused on front desk operations. Receptionists register patients, process admissions and discharges, schedule appointments, assign doctors to patients, generate bills, and record payments. They are the hub of daily operations.

- **Doctor stories** focused on patient care. Doctors need to see their assigned patients, update diagnoses, create treatment plans, prescribe treatments from the catalog, view patient history including disease descriptions, and discharge patients when done. They also need to manage their appointment schedule.

- **Nurse stories** were about daily care. Nurses need to see which patients they are assigned to, record vitals such as blood pressure, temperature, pulse, and oxygen level, and be aware of treatment changes. We originally wanted real-time notifications for treatment updates but had to settle for the nurse refreshing the page or checking

the patient detail view, since WebSockets were too much for our timeline.

- **Ward Boy stories** were about logistics. They need to see which rooms need cleaning after discharge, transport patients between rooms when requested, and mark transport tasks as completed. We almost cut this role to save time but kept it because it makes the room workflow more realistic.

- **Patient stories** were lower priority but we implemented them anyway. Patients should be able to view their own records, see their bill status, book appointments, and receive updates. We figured this adds professionalism and gives patients transparency.

2.3 Functional Requirements

We broke the system into modules based on the user stories. Here is what each module needs to do.

- **User Management** handles registration, login, logout, and role assignment. Registration requires username, password, and role selection. The system validates username uniqueness, hashes passwords with BCrypt, and issues JWT tokens. We support six roles with different permissions enforced at both the URL level and the method level. Admins can view all users, edit details, change roles, reset passwords, and deactivate accounts.

- **Patient Management** lets staff register patients with personal details, emergency contacts, blood group, disease description, current treatment notes, and status. Patients can be updated but not fully deleted. We implemented soft delete by marking is_active as false because medical records need to be preserved for legal reasons. Search works by name, phone, or patient ID, and results show admission history and current vitals.

- **Doctor Management** stores doctor profiles with specialization, qualification, experience years, consultation fee, and availability status. Doctors can be assigned to patients through the doctor_patient junction table, which tracks whether the assignment is primary and when it started.

- **Staff Management** covers nurses, ward boys, and receptionists. Staff have roles, assigned wards, shifts, and availability status. The admin can add, edit, or remove staff members. Receptionists are created through a separate admin function to ensure proper role assignment.

- **Room and Admission Management** tracks rooms by number, type (General, ICU, Operation Theater), floor, capacity, current occupancy, daily rate, and status. We prevent overbooking through application logic. When a patient is admitted, the room's current occupancy increases. When a patient is discharged, it decreases. If occupancy reaches capacity, the room status changes to OCCUPIED. Admissions link a patient and a room, tracking admission date, discharge date, total days, status, and notes.

- **Appointment Management** lets patients and receptionists schedule appointments. Appointments have a date, specialization, status (Pending, Completed, Cancelled), and notes. Receptionists can assign doctors to pending appointments, and the system checks for scheduling conflicts within a 30-minute window. Completed appointments can be billed.

- **Treatment Management** has two parts: the treatments catalog (name, category, unit cost, description) and treatment records (which patient received what treatment, from which doctor, administered by which staff member, when, and in what quantity). The unit_cost_snapshot field preserves the historical price at the time of treatment so that future catalog price changes do not affect past bills.

- **Vitals Tracking** lets nurses record patient vitals: blood pressure, temperature, pulse, and oxygen level. The system validates physical limits (temperature between 25 and 45 degrees Celsius, pulse between 0 and 300, oxygen between 0 and 100 percent).

- **Transport Tasks** lets staff request patient transport between rooms. Tasks have a status (Pending, Accepted, In Progress, Completed, Cancelled), assigned staff member, from room, to room, and completion time. Ward boys can accept and complete tasks.

- **Billing** automatically generates bills when a patient is discharged or an outpatient appointment is completed. It calculates room charges from stay duration and daily rate, sums treatment costs using the snapshot prices, adds doctor consultation fees from all assigned doctors, adds

outpatient charges for completed appointments, applies discounts and tax, and produces a total. Bills track payment status (Pending, Partial, Paid, Refunded) and paid amount. Admins can apply discounts, void bills, and process refunds. Bills can be exported as PDF.

- **Dashboard and Reporting** shows aggregated statistics: active admissions, available rooms, active doctors, total revenue, room occupancy by type, and bill summaries. We use Chart.js for visualizations.

2.4 Non-Functional Requirements

We aimed for page loads under 3 seconds and search results within 1 second for our test dataset of around 100-150 records. The dashboard should render charts within 3 seconds. We are not building for a massive scale, maybe 50 concurrent users max, which is realistic for a small hospital.

Security was non-negotiable. Passwords must be hashed with BCrypt (strength 10). All inputs validated on both client and server. SQL injection prevented through JPA parameterized queries. CSRF tokens on all state-changing forms. Role checks before every sensitive operation. Session timeout after 30 minutes of inactivity.

For usability, the interface needs to work on desktop, tablet, and mobile. We implemented a custom responsive grid using CSS Flexbox/Grid and a collapsible sidebar controlled by CSS state management for mobile. Forms

show inline validation. Destructive actions have confirmation dialogs. Async operations show loading spinners so users know something is happening.

Reliability means graceful error handling. Users should never see raw stack traces. Database transactions protect multi-step operations like billing. We also used localStorage on some long forms so data is not lost if the browser closes accidentally.

3. System Architecture

3.1 Architecture Overview

Our application follows a client-server architecture with a clear separation between the frontend SPA and the backend REST API. We did not use server-side rendering because we wanted the frontend to be a true single-page application (SPA).

- **The Presentation Layer** is entirely client-side. It consists of two main HTML pages: auth.html for login and registration, and index.html for the main dashboard. All routing and view switching is handled by JavaScript. We wrote modular JavaScript files for each feature area (admin.js, admissions.js, appointments.js, billing.js, patients.js, etc.) that communicate with the backend via fetch API calls. CSS is fully custom, using CSS variables for theming and glassmorphism effects for a

modern look. We support both light and dark themes, with the preference stored in localStorage.

- **The Business Logic Layer** is where our Spring Boot controllers and services live. Controllers are simple: they receive HTTP requests, validate JWT tokens, call services, and return JSON responses. Services contain the actual business rules: checking room availability, calculating bills, validating appointment conflicts, and enforcing role permissions. We tried to keep business logic out of controllers because it makes testing easier and the code cleaner, though some controllers ended up more complex than intended due to time constraints.

- **The Data Access Layer** uses Spring Data JPA. We define repository interfaces extending JpaRepository, and Spring generates the basic queries automatically. For complex queries, like dashboard statistics or multi-criteria search, we wrote custom JPQL or native SQL queries. Hibernate handles the object-relational mapping (ORM) and generates our database schema from the entity classes.

MySQL sits at the bottom, running on Railway's managed database service. Hibernate connects to it through the connection pool configured in application.properties. We use ddl-auto=update during development so schema changes are applied automatically.

A typical data flow looks like this: a user submits an admission form. The JavaScript POSTs JSON to /api/v1/admissions with a Bearer token in the header. The JwtAuthenticationFilter validates the token and sets the security context. The AdmissionController validates the DTO with @Valid.

AdmissionService checks room capacity, increments occupancy, and builds the Admission entity. The repository saves it to MySQL. RoomRepository updates the current occupancy. The controller returns a JSON response, and the JavaScript updates the UI accordingly.

3.2 Technology Stack Justification

Backend: Spring Boot 4.0.6 with Java 21+

- We picked Spring Boot because our instructor said it is the industry standard and more productive than raw servlets. After doing the servlet labs earlier in the course, we agreed. Writing servlets manually involves so much boilerplate for database connections, routing, and security that you spend more time on infrastructure than actual features. Spring Boot handles dependency injection, auto-configuration, and security setup, which let us focus on hospital logic.
- We did briefly consider Node.js with Express since one of us knows JavaScript, but the other person is much stronger in Java. Learning Express syntax, middleware, and async patterns while also building a complex system in five weeks seemed risky. We also looked at Jakarta EE but the XML configuration looked painful compared to Spring Boot's convention-over-configuration approach. So Spring Boot it was.
- We recommended using Java 21+ LTS version and Spring Boot 4.0.6. Features like pattern matching and improved garbage collection were

nice to have, though we did not use advanced language features extensively.

Frontend: HTML, CSS, and JavaScript

- We chose a single-page application (SPA) approach with vanilla JavaScript because we wanted to demonstrate deep understanding of DOM manipulation, fetch API, and asynchronous programming without hiding behind a framework. The frontend is completely decoupled from the backend: it calls REST APIs and renders JSON responses dynamically. This architecture is closer to modern industry practices where frontend and backend are separate concerns.
- We did not use Thymeleaf because we did not want server-side rendering. We did not use Bootstrap because we wanted a custom design system with CSS variables, glassmorphism, and a dark mode toggle. Writing our own CSS was more work but gave us a unique look that does not resemble every other Bootstrap site.
- For a real product we might use React or Vue, but for a 5-week academic group project, vanilla JS was the smarter choice because it excluded build tooling complexity and let us focus on main features.

Database: MySQL

- We planned on using MongoDB; however, the instructor recommended MySQL and luckily, we had used it in the Principles of Database Management course last semester. We are comfortable with MySQL Workbench for designing schemas and writing queries. For a student

project with approximately 100 records, MySQL is perfectly capable. We know MongoDB might be a good option, but we would follow our instructor's advice.

PDF Generation: iText 5.5.13.3

- We use iText for PDF bill generation because it is a library with extensive formatting capabilities. Our PDFs include styled tables, colored headers, alternating row backgrounds, and professional typography. We did not use OpenPDF because we found more examples and documentation for iText.

JWT Authentication: jjwt 0.11.5

- We chose JWT for stateless authentication because it fits our SPA architecture perfectly. The frontend stores the token in localStorage or sessionStorage and sends it with every request. This eliminates server-side session management and makes the API truly stateless.

Hosting: Railway

- We deployed on Railway because it is designed for quick deployments without server management. It supports Java applications and manages MySQL. We looked at Heroku too, but Railway felt more modern. AWS was too complex for our current skill level.

Build Tool: Maven

- Maven is the default for Spring Boot projects and we already knew it from previous labs of the course. We saw no reason to switch to Gradle.

3.3 Design Patterns

We used a few patterns, some intentionally and some because Spring Boot only allows them.

- **MVC (Model-View-Controller):** Our models are JPA entities, views are the JSON responses consumed by the frontend, and controllers handle HTTP requests. We added a Service layer between controller and repository, so technically it is more like Model-View-Controller-Service, but the MVC structure is still the backbone.
- **Repository Pattern:** Each entity has a repository interface. Services call repository methods without knowing SQL. This abstraction means if we ever switched databases, the service layer would not change.
- **Dependency Injection:** Spring's IoC container wires everything together. We use constructor injection, which is better than field injection because it makes dependencies explicit and helps with unit testing.
- **DTO (Data Transfer Object):** For complex forms like admission creation or patient registration, we use DTOs instead of passing entities directly. This prevents exposing internal entity structure and lets us

validate the form as a whole. For example, AdmissionRequest carries patientId and roomId, and RegisterRequest carries all patient registration fields.

- **Singleton:** Spring beans are singletons by default. Our services and repositories exist as single instances, which is efficient as long as we do not store request specific state in them.

3.4 API Design

Our frontend communicates with the backend entirely through REST APIs. All endpoints are prefixed with /api/v1/ for versioning. We have /api/v1/auth, /api/v1/admin, /api/v1/admissions, /api/v1/appointments, /api/v1/bills, /api/v1/doctors, /api/v1/doctor-patient, /api/v1/patients, /api/v1/rooms, /api/v1/staff, /api/v1/treatments, /api/v1/treatment-records, /api/v1/transport-tasks, /api/v1/users, and /api/v1/vitals.

HTTP methods: GET for reading, POST for creating, PUT for full updates, PATCH for partial updates, and DELETE for soft deletes. We rarely perform true deletes because of data preservation requirements.

Status codes: 200 for success, 201 for creation, 400 for bad input, 401 for not logged in, 403 for wrong role, 404 for missing resources, 409 for conflicts like duplicate entries, 500 for unexpected server errors. We log 500s with stack traces in development but only show a generic message in production.

For validation errors, we use @ControllerAdvice to catch exceptions and return structured JSON with field-level messages. The frontend JavaScript parses this and shows inline errors or toast notifications.

Example Request/Response:

Login:

```
POST /api/v1/auth/login

Content-Type: application/json



{

  "username": "admin",

  "password": "Pass1234",

  "rememberMe": false

}
```

Response:

```
HTTP/1.1 200 OK



{

  "token": "eyJhbGciOiJIUzI1NiIs...",

  "role": "ADMIN",

  "username": "admin"

}
```

Create Admission:

```
POST /api/v1/admissions

Authorization: Bearer <token>

Content-Type: application/json



{

  "patientId": "PAT-2024-0001",

  "roomId": 2,

  "notes": "Emergency appendicitis"

}
```

Response:

```
HTTP/1.1 200 OK



{

  "admissionId": 64,

  "patient": { ... },

  "room": { ... },

  "admissionDate": "2026-05-29T14:30:00",
```

```
  "status": "ACTIVE"

}
```

4. Database Design

4.1 Relational Schema

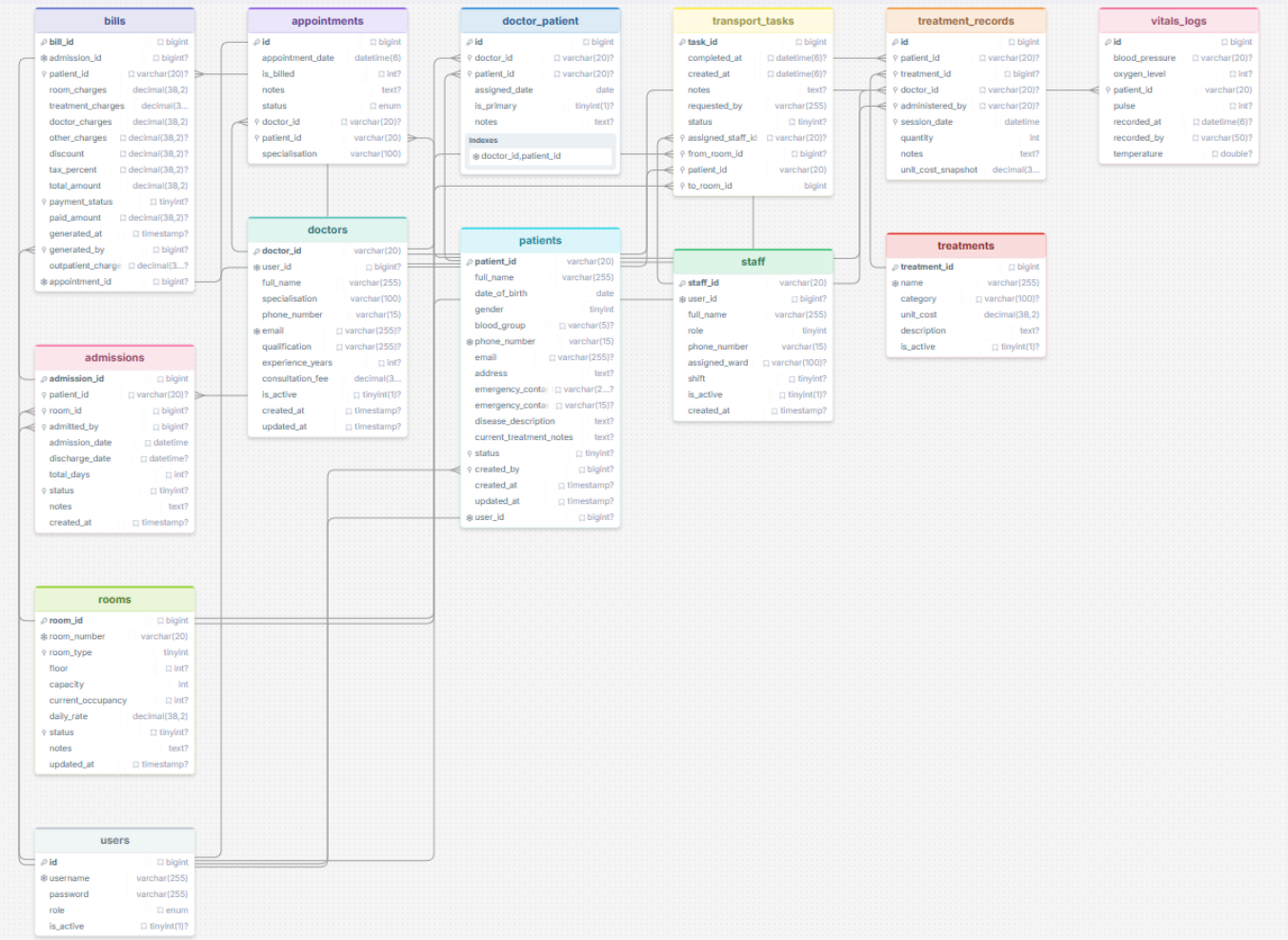

Figure 4.1.1: Relational Schema Diagram

4.2 Table Schemas

Here are the detailed schemas based on our MySQL Workbench design. We used varchar(20) for business entity IDs (patients, doctors, staff) and bigint auto-increment for system generated IDs.

users - stores authentication info.

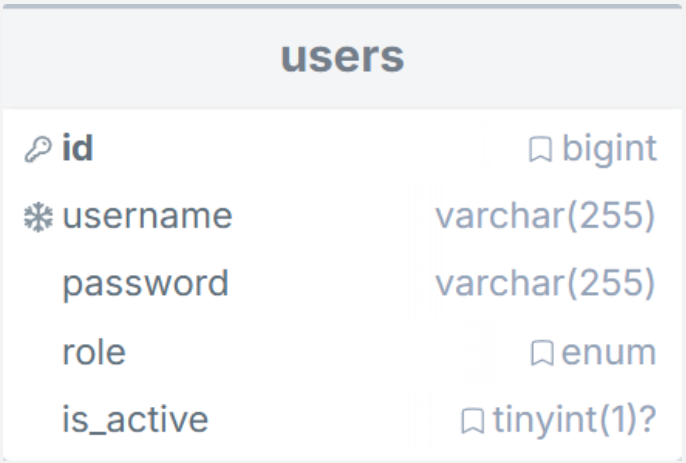

Figure 4.2.1: users table

patients - stores medical and personal info.

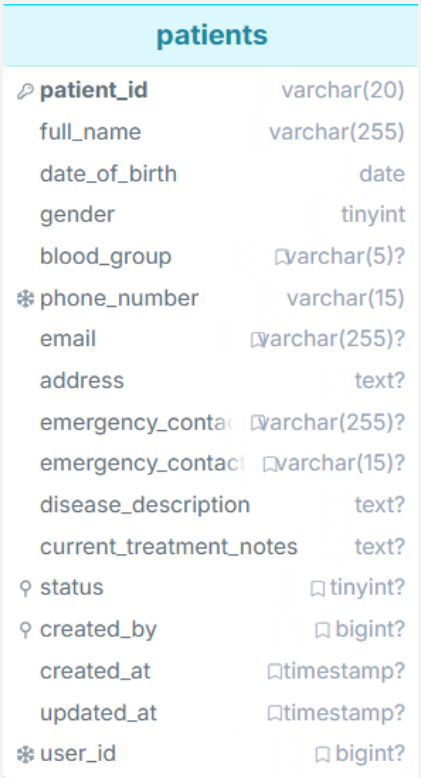

Figure 4.2.2: patients table

rooms - tracks physical rooms.

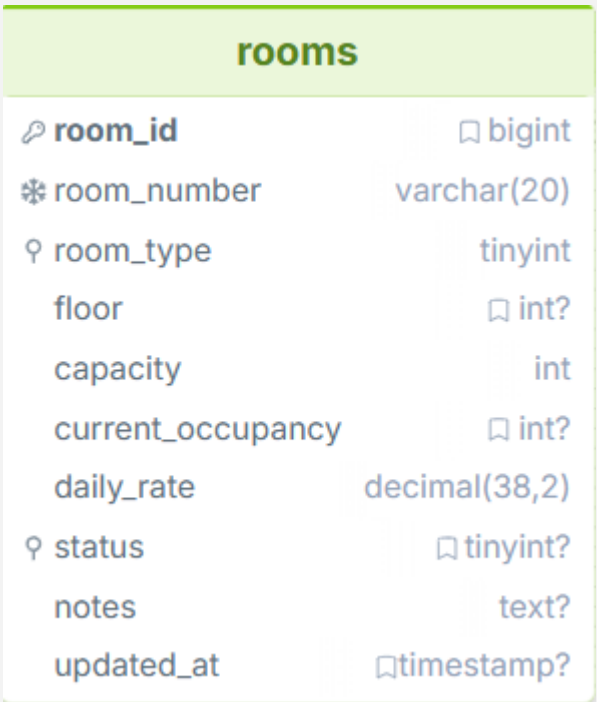

Figure 4.2.3: rooms table

admissions - central table for patient stays.

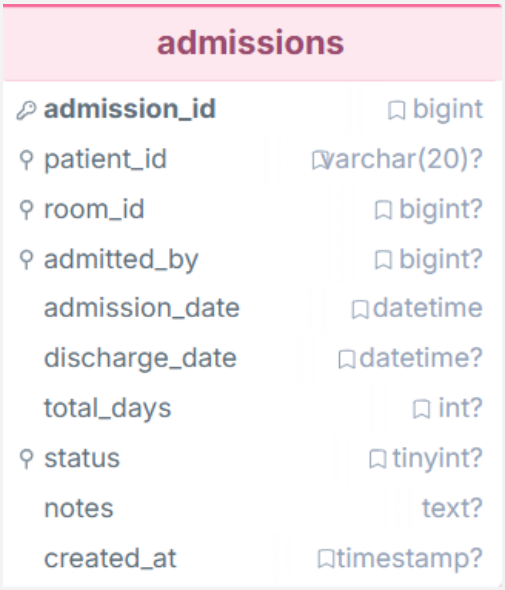

Figure 4.2.4: admissions table

bills - stores calculated charges.

Figure 4.2.5: bills table

appointments - outpatient visits.

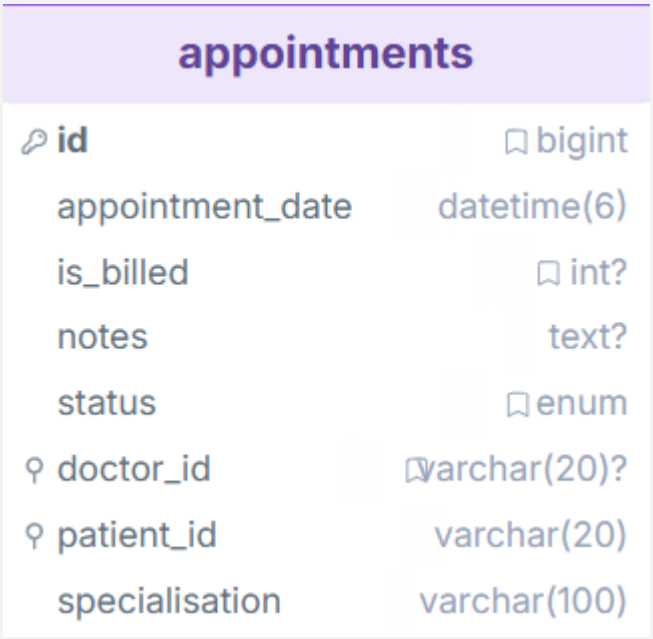

Figure 4.2.6: appointments table

doctor_patient - junction table for assignments.

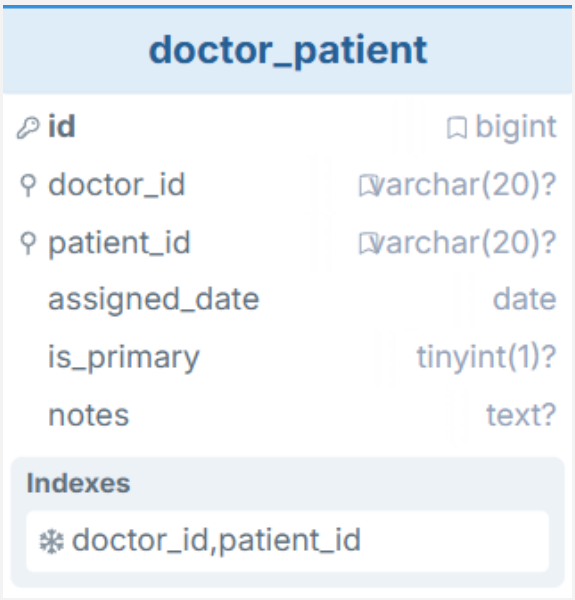

Figure 4.2.7: doctor_patient table

treatments - catalog of available treatments.

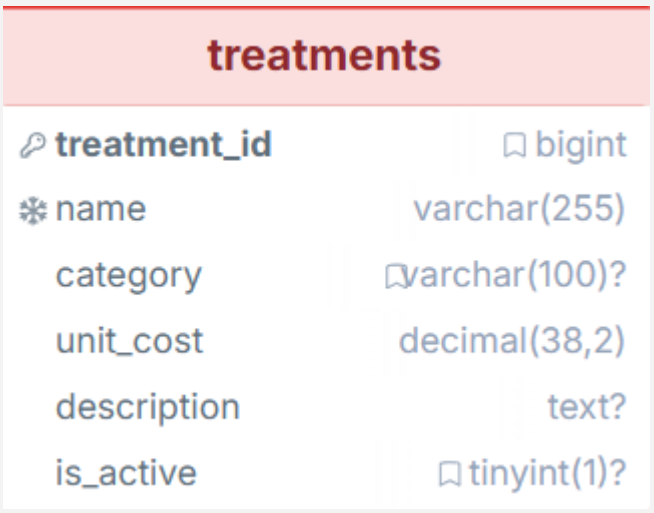

Figure 4.2.8: treatments table

treatment_records - actual treatments given to patients.

Figure 4.2.9: treatment_records table

vitals_logs - patient vital signs.

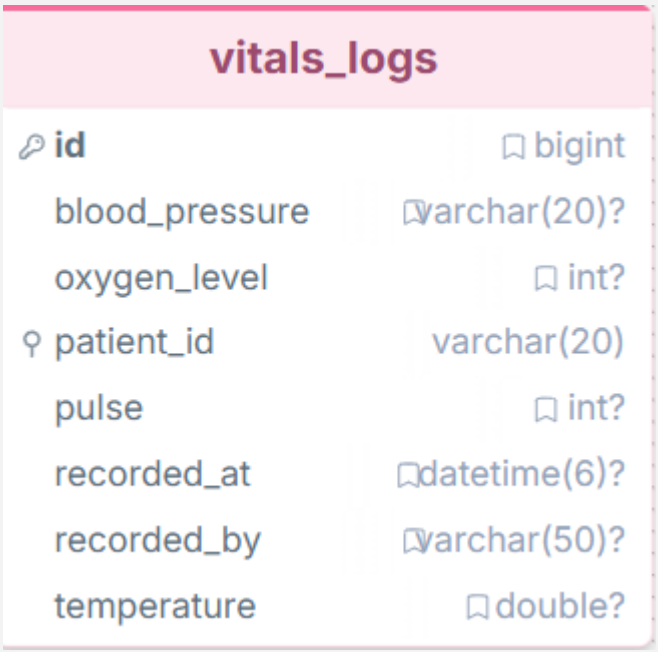

Figure 4.2.10: vitals_logs table

transport_tasks - patient movement between rooms.

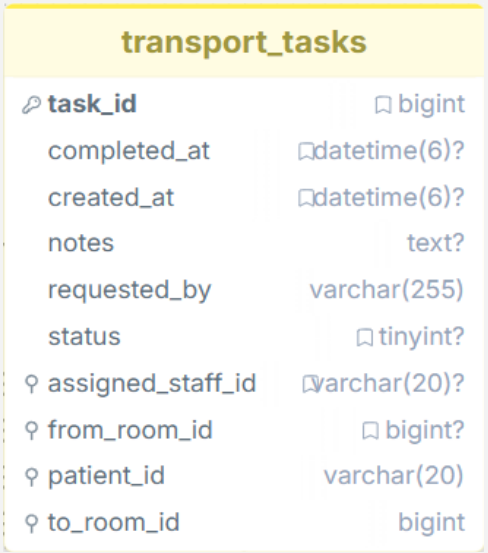

Figure 4.2.11: transport_tasks table

4.3 Relationships Explanation

One-to-One:

- Users to Patients: One user account links to one patient profile via user_id. We separated authentication from medical data so a person could theoretically have both a staff and patient account without mixing data.
- Users to Doctors: Similarly, one doctor has one user account for login.
- Users to Staff: One staff member has one user account.
- Admission to Bill: One admission generates exactly one bill. We enforce this through a unique constraint on bills.admission_id.
- Appointment to Bill: One appointment can generate exactly one bill, enforced by unique constraint on bills.appointment_id.

One-to-Many:

- Patient to Admissions: One patient can be admitted multiple times over their lifetime.
- Room to Admissions: One room can host many admissions sequentially. We track current occupancy to prevent overbooking.
- Doctor to Appointments: One doctor can have many scheduled appointments.
- Patient to Appointments: One patient can book multiple appointments.

- Patient to TreatmentRecords: One patient can receive many treatments over time.
- Treatment to TreatmentRecords: One treatment from the catalog can be applied to many patients.
- Doctor to TreatmentRecords: One doctor can prescribe many treatments.
- Staff to TreatmentRecords: One staff member can administer many treatments.
- Patient to VitalsLogs: One patient has many vital sign recordings.
- Patient to TransportTasks: One patient can be moved multiple times.
- Staff to TransportTasks: One staff member can be assigned multiple transport tasks.

Many-to-Many:

- Doctors and Patients connect through DoctorPatient, which tracks assignment date, whether it is the primary doctor, and notes. This lets multiple doctors treat one patient, and one doctor treat multiple patients.

Bills linking to multiple parents:

- Bills can reference either an admission (for inpatient) or an appointment (for outpatient). We use admission_id and appointment_id as nullable foreign keys, with application logic ensuring exactly one is set. This avoids creating separate bill tables for inpatient and outpatient.

4.4 Normalization Analysis

We aimed for Third Normal Form. For 1NF, all columns are atomic which is no repeating groups or multi-valued attributes. For 2NF, since every table uses a single column primary key (either surrogate bigint or business varchar), there can not be partial dependencies. Every attribute depends on the full primary key.

For 3NF, we checked for transitive dependencies. In users, nothing depends on user_name except the id. In patients, emergency contacts and disease descriptions are stored with the patient because they are specific to them as we did not create separate tables because that would be over-normalization for this scope.

Intentional denormalization:

- The bills table stores calculated values (room_charges, treatment_charges, doctor_charges, total_amount, total_days) rather than computing them on demand. We did this for two reasons. Firstly, room rates and treatment prices might change over time, but a historical bill should preserve the prices at the time of service. Secondly, computing totals by joining multiple tables every time someone views a bill would be slow. By storing the snapshot at generation time, we ensure the bill is immutable and fast to retrieve. This is standard practice in billing systems.

- The treatment_records table stores unit_cost_snapshot for the same reason — if the treatment catalog price changes later, the historical record preserves what was actually charged at the time of service.

4.5 Indexing Strategy

We indexed foreign keys and frequently queried columns:

- **users.user_name:** for login lookups
- **patients.user_id:** unique index for the one-to-one relationship
- **patients.phone_number, patients.email:** for search
- **doctors.user_id:** unique index
- **staff.user_id:** unique index
- **admissions.patient_id, admissions.room_id:** foreign key indexes
- **admissions.status**: for filtering active admissions
- **bills.patient_id, bills.admission_id, bills.appointment_id:** for bill lookups
- **bills.payment_status:** for filtering unpaid bills
- **appointments.doctor_id, appointments.patient_id:** for schedule lookups
- **appointments.appointment_date:** for date-range queries
- **doctor_patient.doctor_id + patient_id:** composite index

- **treatment_records.patient_id, treatment_records.treatment_id, treatment_records.doctor_id:** foreign key indexes
- **vitals_logs.patient_id + recorded_at:** composite index for recent vitals
- **transport_tasks.patient_id, transport_tasks.assigned_staff_id, transport_tasks.status:** for task filtering

We did not index everything because indexes slow down writes and take disk space. We focused on columns used in JOINs, WHERE clauses, and ORDER BY operations.

5. Implementation Details

5.1 UML Diagram

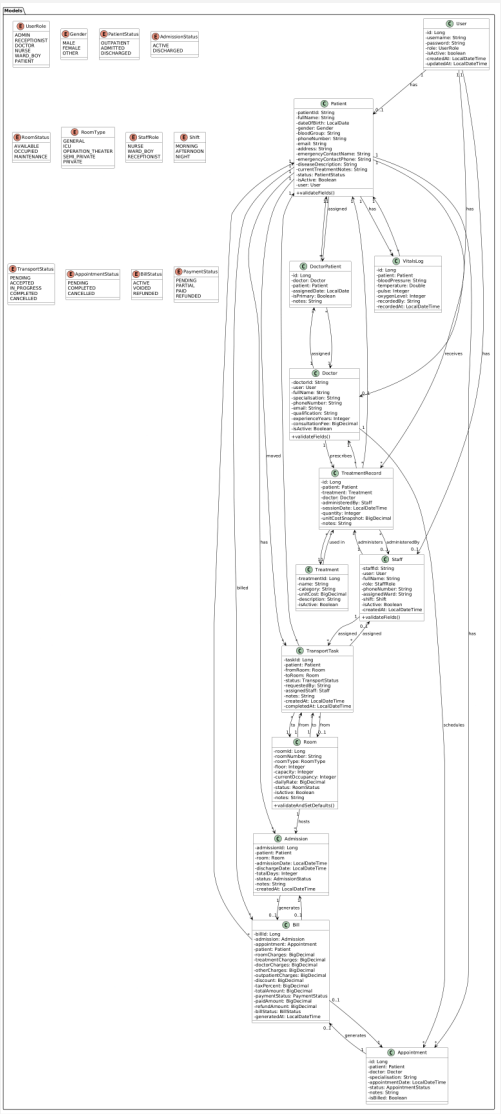

Figure 5.1.1: Models module

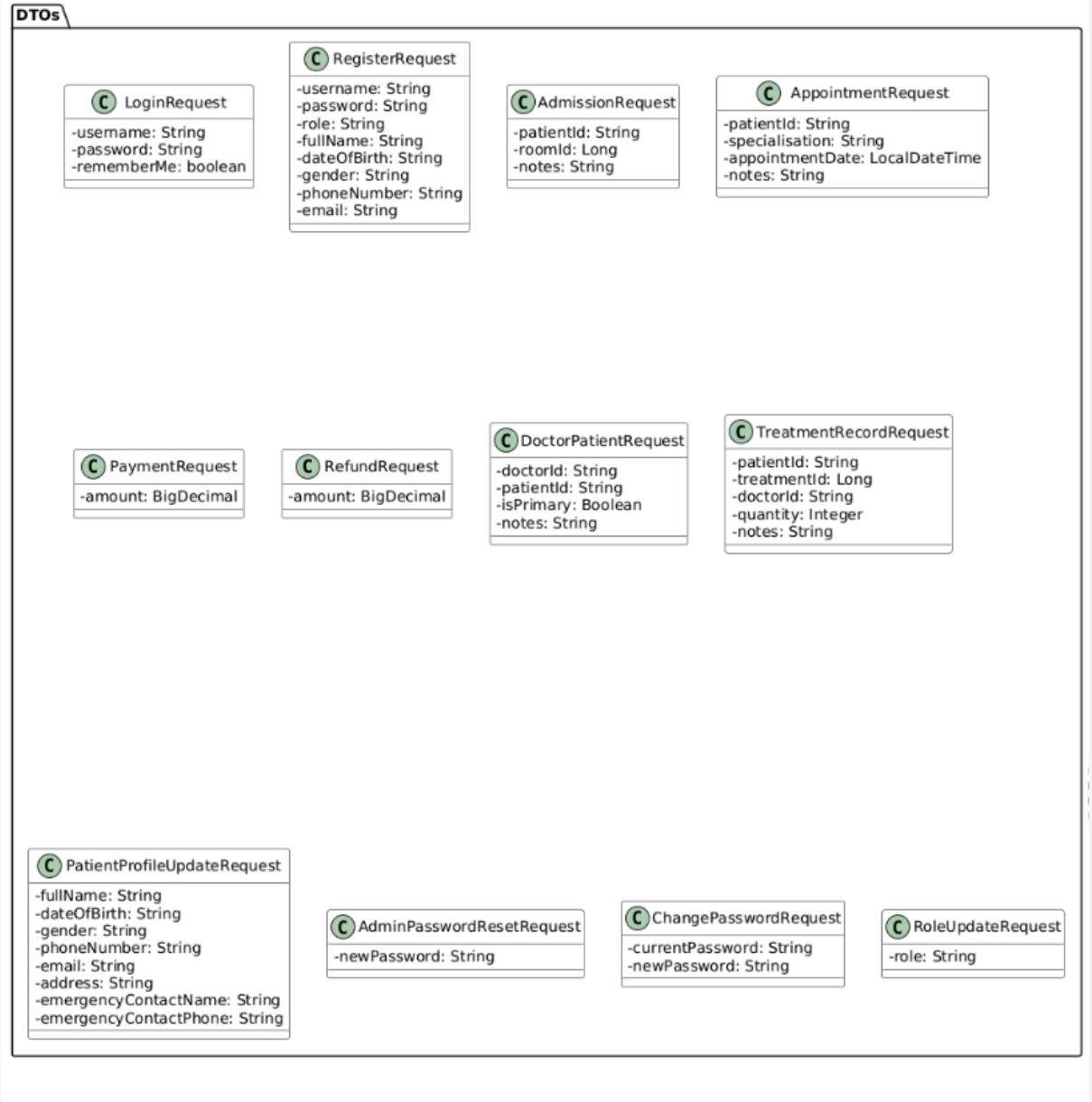

Figure 5.1.2: DTOs module

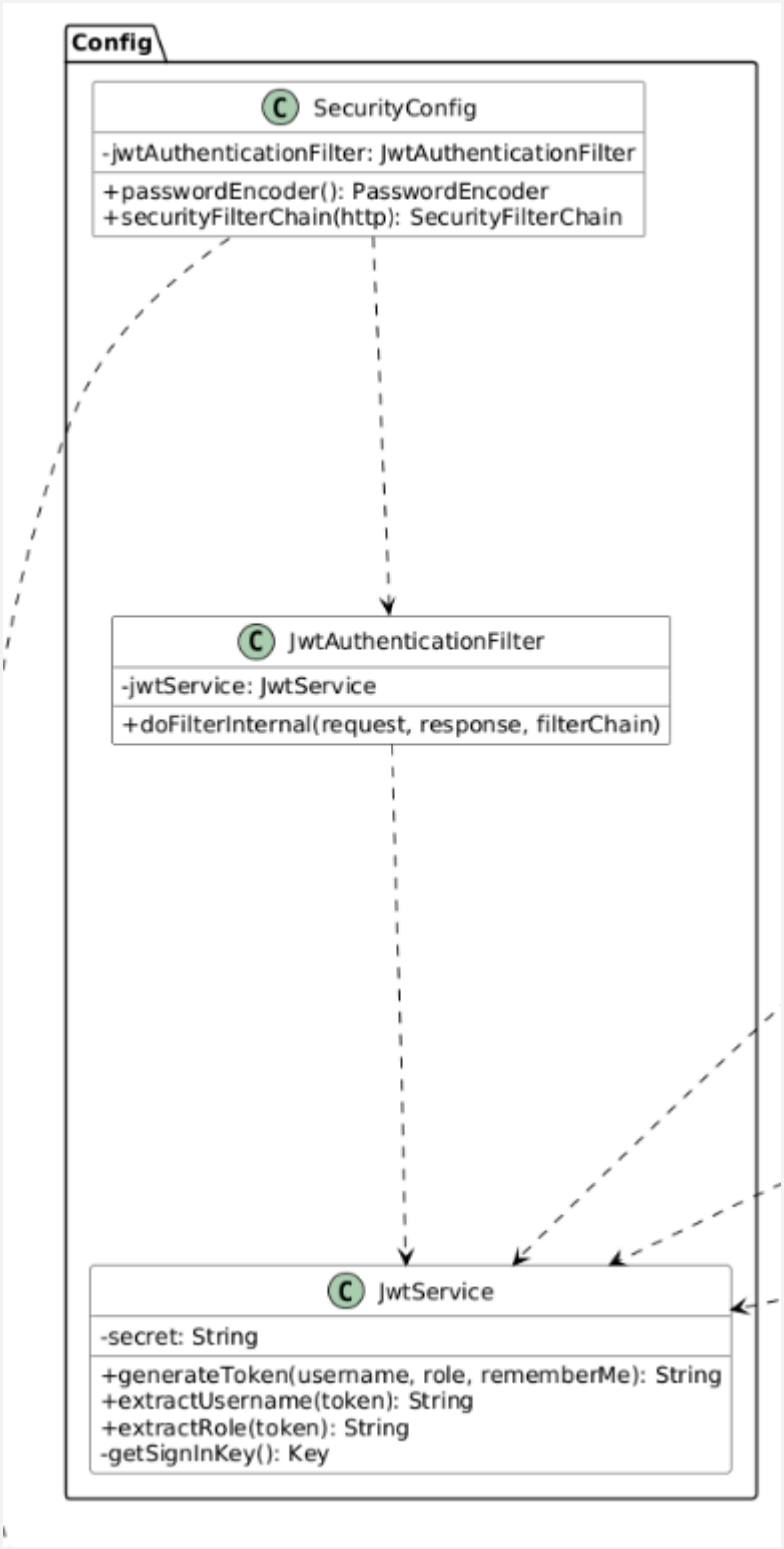

Figure 5.1.3: Config module

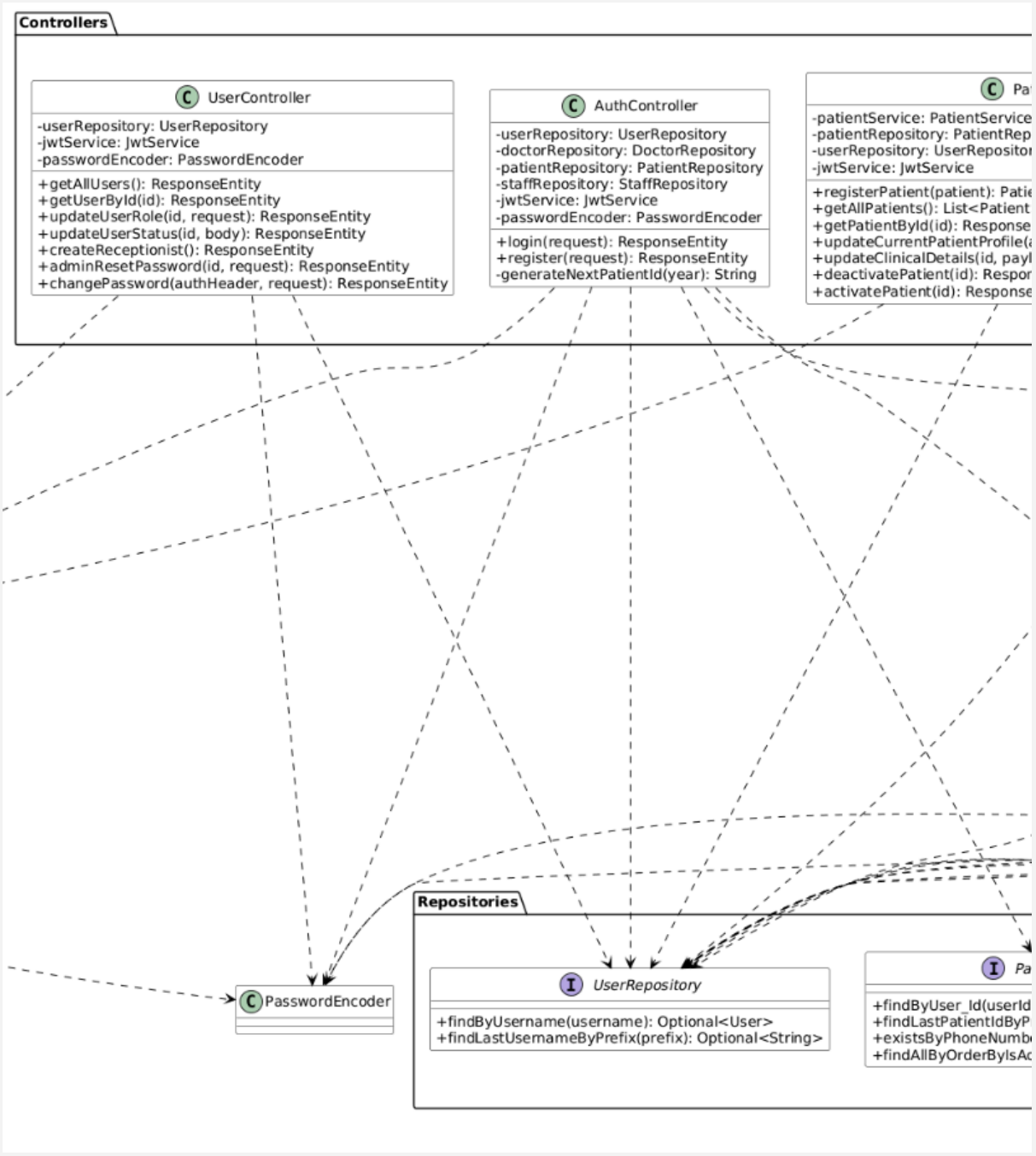

Figure 5.1.4: Controller and Repositories modules

5.2 Key Algorithms and Business Logic

Room Availability and Admission:

- When admitting a patient, we verify the room has capacity. We check current_occupancy against capacity. If there is space, we increment current_occupancy and create the admission. On discharge, we decrement current_occupancy. If occupancy reaches capacity, the room status changes to OCCUPIED. If it drops below capacity, it returns to AVAILABLE (unless under maintenance). The whole operation is wrapped in @Transactional so that if the admission save fails, the room occupancy does not get stuck.

```
@Transactional
public Admission admitPatient(AdmissionRequest request) {
      Patient                          patient                          =
patientRepository.findById(request.getPatientId())
      .orElseThrow(() -> new RuntimeException("Patient not found"));

      if (patient.getStatus() == PatientStatus.ADMITTED) {
      throw new RuntimeException("Patient is already admitted");
      }

      Room room = roomRepository.findById(request.getRoomId())
      .orElseThrow(() -> new RuntimeException("Room not found"));

      if (room.getStatus() == RoomStatus.MAINTENANCE) {
      throw new RuntimeException("Room is under maintenance");
      }
      if (room.getCurrentOccupancy() >= room.getCapacity()) {
      throw new RuntimeException("Room is at full capacity");
```

```
        }

        Admission admission = new Admission();
        admission.setPatient(patient);
        admission.setRoom(room);
        admission.setAdmissionDate(LocalDateTime.now());
        admission.setStatus(AdmissionStatus.ACTIVE);

        room.setCurrentOccupancy(room.getCurrentOccupancy() + 1);
        if (room.getCurrentOccupancy() >= room.getCapacity()) {
        room.setStatus(RoomStatus.OCCUPIED);
        }

        patient.setStatus(PatientStatus.ADMITTED);
        patientRepository.save(patient);
        roomRepository.save(room);

        return admissionRepository.save(admission);
}
```

Automated Bill Calculation:

- This was probably the trickiest part. The bill needs to sum room charges, treatment costs, doctor costs, and outpatient costs. Room charges are calculated as total_days multiplied by daily_rate. Treatment charges sum the unit_cost_snapshot multiplied by quantity from all treatment_records for that patient during the admission period. Doctor charges come from the consultation_fee of all assigned doctors. Outpatient charges come from completed appointments. We also apply discount and tax. We used BigDecimal for everything because we

learned in a previous lab that double causes rounding errors with money.

```
public Bill generateBill(Long admissionId) {

	Admission admission = admissionRepository.findById(admissionId)

	.orElseThrow(() -> new RuntimeException("Admission not found"));


	if (admission.getStatus() == AdmissionStatus.ACTIVE) {

	throw new RuntimeException("Patient must be discharged before generating a bill");

	}


	if (billRepository.findByAdmission_AdmissionId(admissionId).isPresent()) {

	throw new RuntimeException("Bill already exists for this admission");

	}


	BigDecimal roomCharges = admission.getRoom().getDailyRate()

	.multiply(BigDecimal.valueOf(admission.getTotalDays()));


	BigDecimal treatmentCharges = treatmentRecordRepository

	.sumTreatmentCostByPatientAndDateRange(

		patientId,

		admission.getAdmissionDate().minusSeconds(1),

		admission.getDischargeDate().plusSeconds(1)

	);

	if (treatmentCharges == null) treatmentCharges = BigDecimal.ZERO;


	List<DoctorPatient> assignments = doctorPatientRepository

	.findByPatient_PatientId(patientId);

	BigDecimal doctorCharges = assignments.stream()

	.map(dp -> dp.getDoctor().getConsultationFee())
```

```
        .reduce(BigDecimal.ZERO, BigDecimal::add);

        List<Appointment> completedAppointments = appointmentRepository
        .findByPatient_PatientIdAndStatusAndIsBilled(
              patientId, AppointmentStatus.COMPLETED, false
        );
        BigDecimal outpatientCharges = completedAppointments.stream()
        .map(app -> app.getDoctor().getConsultationFee())
        .reduce(BigDecimal.ZERO, BigDecimal::add);

        BigDecimal subtotal = roomCharges.add(treatmentCharges)
        .add(doctorCharges).add(outpatientCharges);
        BigDecimal discountFactor = BigDecimal.ONE.subtract(
        discount.divide(BigDecimal.valueOf(100), 4, RoundingMode.HALF_UP)
        );
        BigDecimal afterDiscount = subtotal.multiply(discountFactor);
        BigDecimal taxFactor = BigDecimal.ONE.add(
        taxPercent.divide(BigDecimal.valueOf(100),                    4,
RoundingMode.HALF_UP)
        );
        BigDecimal total = afterDiscount.multiply(taxFactor);

        Bill bill = new Bill();
        bill.setAdmission(admission);
        bill.setPatient(patient);
        bill.setRoomCharges(roomCharges);
        bill.setTreatmentCharges(treatmentCharges);
        bill.setDoctorCharges(doctorCharges);
        bill.setOutpatientCharges(outpatientCharges);
        bill.setTotalAmount(total.setScale(2, RoundingMode.HALF_UP));
        bill.setPaymentStatus(PaymentStatus.PENDING);

        return billRepository.save(bill);
```

```
}
```

Appointment Conflict Detection:

- When assigning a doctor to an appointment, we check if the doctor already has another appointment within a 30-minute window. This prevents double-booking and ensures doctors have enough time between patients.

```
public Appointment assignDoctor(Long id, String doctorId) {

	Appointment appointment = appointmentRepository.findById(id)

	.orElseThrow(() -> new RuntimeException("Appointment not found"));



	LocalDateTime start = appointment.getAppointmentDate().minusMinutes(30);

	LocalDateTime end = appointment.getAppointmentDate().plusMinutes(30);



	boolean isBusy = appointmentRepository


.existsByDoctor_DoctorIdAndStatusNotAndAppointmentDateAfterAndAppointmentDateBefore(

		doctorId, AppointmentStatus.CANCELLED, start, end

	);



	if (isBusy) {

	throw new RuntimeException("Doctor already has an appointment around this time");

	}



	appointment.setDoctor(doctor);

	return appointmentRepository.save(appointment);

}
```

5.3 Security Implementation

Password Hashing: We use BCryptPasswordEncoder with the default strength. Passwords are hashed in the AuthController before reaching the service. We never store or log plain text passwords. The User entity uses @JsonIgnore on the password field so it is never returned in API responses.

SQL Injection: All database access goes through Spring Data JPA, which uses prepared statements. For native queries, we use named parameters. We never concatenate user input into SQL strings.

XSS Prevention: Since we use a SPA, XSS prevention happens in two places. On the backend, we validate and sanitize inputs. To ensure security while maintaining flexibility on the frontend, we implement a hybrid approach: static UI structures are rendered via innerHTML, while user-provided data is strictly injected using textContent.

JWT Authentication: We use the jjwt library for token generation and validation. The JwtAuthenticationFilter intercepts every request, extracts the Bearer token from the Authorization header, validates the signature and expiration, and sets the authentication in the SecurityContext. Tokens contain the username and role. We do not use server-side sessions; the application is completely stateless.

CSRF: We disabled CSRF protection because we use JWT tokens rather than session cookies. CSRF is only relevant for cookie-based sessions. Since our frontend sends the JWT in the Authorization header, CSRF attacks are not possible.

Role-Based Access Control: We configured Spring Security with URL-based restrictions and method-level annotations. For example, /api/v1/admin/** requires ADMIN role, and /api/v1/admissions POST requires ADMIN or RECEPTIONIST. We also use @PreAuthorize on controller methods for finer control. Some endpoints have custom logic to ensure patients can only access their own data.

5.4 Error Handling Strategy

On the frontend, we have a global error handler for fetch requests that shows toast notifications. Form validation uses CSS classes for valid and invalid states. Server validation errors return JSON with field-level messages that JavaScript displays inline or as alerts.

On the backend, @ControllerAdvice catches exceptions globally. We have custom exceptions: ResourceNotFoundException (404), RuntimeException (400). Each maps to a specific HTTP status and message. We log stack traces in development but only show generic "Something went wrong" messages in production, with a reference ID for debugging.

For database errors, we catch DataIntegrityViolationException and translate them. For example, duplicate username throws "Username is already taken", duplicate phone number throws "Phone number is already registered", and foreign key violations throw "Cannot complete action because the record is referenced elsewhere".

5.5 Advanced Features Implementation

Dashboard with Chart.js:

- The admin dashboard shows statistics via Chart.js. The backend endpoint /api/v1/admin/dashboard returns aggregated data: active admissions count, available rooms count, active doctors count, total revenue from paid bills, room occupancy by type, and bill summaries for charts. We used MySQL GROUP BY and COUNT/SUM queries. The frontend renders bar charts and pie charts from this data.

PDF Bill Export:

- We use iText 5.5.13.3 to generate professional PDF bills. The BillPdfService creates a document with hospital branding, patient details, stay information, an itemized table of all charges (room, treatments, doctor consultations, outpatient visits), and a summary box showing subtotal, discount, tax, grand total, paid amount, and balance due. The PDF uses custom fonts, colored headers, alternating row backgrounds, and a footer. We stream the PDF directly to the browser as a download.

Automated Billing with Partial Payments and Refunds:

- Our billing system is more advanced than a simple invoice generator. It supports partial payments, where a patient can pay in installments and the system tracks the remaining balance. Admins can apply percentage

discounts before payment, which recalculates the total. If a mistake is made, admins can void a bill (marking it inactive) or process a refund up to the paid amount. The bill status tracks whether it is ACTIVE, VOIDED, or REFUNDED.

6. Session management

6.1 Purpose

In a collaborative medical environment like Hospitalz, secure and reliable session management is crucial. Medical records, patient clinical histories, and billing data are highly sensitive. We must strictly control who has access to this information.

Since we designed our system with a clear separation between the Spring Boot backend and the web frontend, we needed a session mechanism that did not depend on heavy server-side state. The session management system has several responsibilities:

- Maintaining Login State: This allows users to navigate the platform without having to log in again on every page refresh.
- Preventing Unauthorized Access: It ensures that API endpoints, such as those managing prescriptions or room allocations, reject requests without a valid, active session.

- Enforcing Role-Based Permissions: This makes certain that a patient cannot view another patient's medical records or access the administrative dashboard.

6.2 Features / Functionalities

Table 8.2.1: Feature Table

Feature	Description
Stateless Token Generation	After a successful login, the server provides a secure JSON Web Token (JWT) that includes the user's username and role. This approach helps to prevent memory overload on the server side.
Remember Me	Users can stay logged in even after browser restarts by extending the token expiration from 24 hours to 30 days, depending on their login choice.
Role-Based Access Control (RBAC)	Restricts API and page access in real-time. Endpoints such as `/api/v1/admin/**` are completely secured for the ADMIN role.
Client-Side Logout	Instantly invalidates the session on the client side by clearing the tokens

	from browser storage, preventing unauthorized reuse.
Token Validation	Every API request is intercepted and validated in real-time to check the token's signature, integrity, and expiration status.

6.3 Workflow / Process

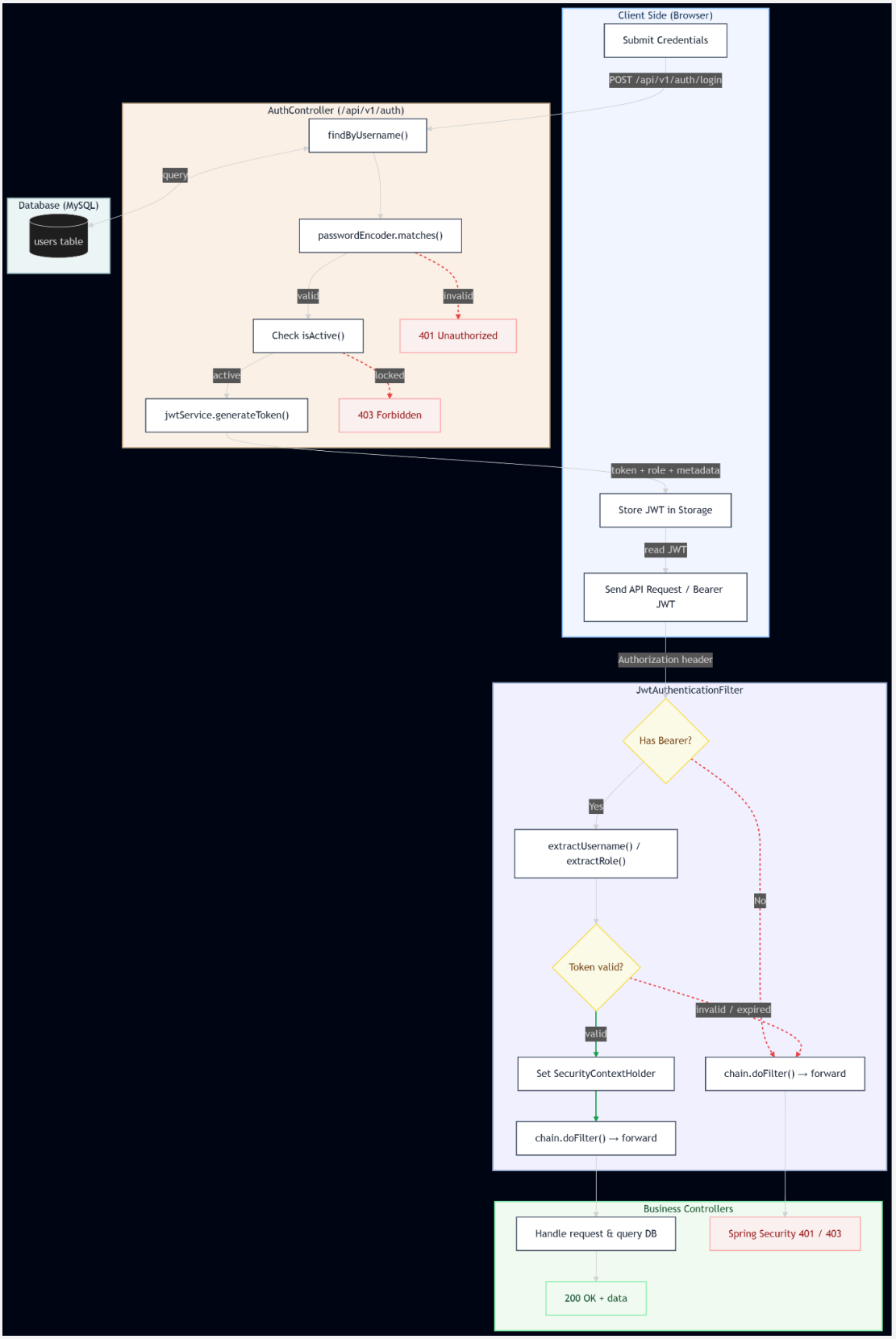

Figure 6.3.1: Workflow / Process

Step-by-Step Execution

The implementation is split between Spring Boot security filters on the backend and storage APIs on the frontend.

1. **User Authentication:** The user enters their username and password. The client sends a POST request to /api/v1/auth/login.
2. **Verification & Token Generation:** AuthController queries the database with findByUsername(), tthen verifies the submitted password against the stored hash with passwordEncoder.matches(). If the account is locked, it immediately returns a 403 Forbidden. If credentials are invalid, it returns a 401 Unauthorized. If both checks succeed, jwtService.generateToken() generates a signed JWT which includes the username and ROLE_<role> claim. The expiry is set to 24 hours (30 days if “Remember Me” is selected).
3. **Client Storage:** The server sends back the JWT, role, username, and a profile identifier (doctorId, patientId, or staffId depending on role). The frontend stores this token in sessionStorage (default) or localStorage (if "Remember Me" was selected).
4. **Resource Access:** The token is added to the Authorization header as Bearer on each subsequent API call.
5. **Request Filtering:** every incoming request is intercepted by JwtAuthenticationFilter. If no Bearer token is found, the request is immediately passed down the filter chain and Spring Security then takes care of access rules. If a token is present, the filter uses jwtService.extractUsername() and jwtService.extractRole() to parse it. If

the token is valid, a UsernamePasswordAuthenticationToken with the extracted role is put into SecurityContextHolder, and the request is forwarded to the target business controller via chain.doFilter(). If the token is invalid or expired, the exception is silently caught, the security context is left empty, and Spring Security automatically returns 401 Unauthorized or 403 Forbidden.

6. **Session Termination:** When the user clicks logout, the frontend clears sessionStorage or localStorage. Since the backend is stateless (no server-side session), removing the token from the client is sufficient to terminate the session immediately.

6.4.1 Backend Authentication Architecture

In order to configure Stateless Session Management with Spring Security 6, we disabled the CSRF protection to make the incorporation of REST clients cleaner, because sessions are stateless:

- SecurityConfig: Configures the filter chain, public assets (like /css/**, /js/**, and /auth.html) are allowed but API routes require authentication.
- JwtAuthenticationFilter: Custom filter extending OncePerRequestFilter. It reads the Authorization header, extracts the token, verifies it and populates the UsernamePasswordAuthenticationToken into the Spring Security Context if it is valid.
- JwtService: Built using the io.jsonwebtoken library. It uses the HS256 signature algorithm with a 256-bit secret key to generate and parse tokens.

6.4.2 Token Longevity Configuration

In JwtService.java the lifetime of the token is set dynamically depending on the user choice in login:

- Standard Session: Expired after 24 hours (86,400,000 ms) of creation.
- Remember Me Session: Expired after 30 days (2,592,000,000 ms) of creation to balance convenience and security.

6.4.3 Frontend Storage Logic

We do not store sensitive credentials in plain. The JWT is stored only in the browser:

```
// From auth.html login handling

const storage = rememberMe ? localStorage : sessionStorage;

storage.setItem("token", data.token);

storage.setItem("role", data.role);

storage.setItem("username", data.username);
```

6.5 Security Considerations

In the design of our sessions, we have incorporated some security best practices to ensure our patients' clinical data is safe:

- Password Hashing: We don't store passwords in plain text. We have adopted a robust one-way hashing scheme with BCrypt and a salt round factor of 10.

- Stateless Protection : Our API endpoints are inherently safe from Cross-Site Request Forgery (CSRF) attacks because we do not use cookie-based sessions. This makes it difficult for attackers to read or spoof the Authorization header.
- Token Verification Integrity: When an attacker tries to change his role claim (e.g., from “PATIENT” to “ADMIN”) in the JWT payload, the cryptographic signature verification will fail, and the backend will silently drop the request.
- Safe Error Handling: JWTAuthenticationFilter ignores and logs silently exceptions from an expired or malformed token, and then continues the filter chain without exposing internal stack traces exposing system information to potential attackers.

6.6 Testing and Validation

Table 8.6.1: Test Cases Table

Test Case	Steps Taken	Actual Result	Status
Valid Login	Entered correct username and password.	Token returned; redirected to dashboard.	PASSED
Invalid Login	Entered incorrect password.	Displays "Wrong username or password" error.	PASSED
Access Unauthenticated	Attempted to fetch `/api/v1/patie`	Blocked by Spring Security, returns 403.	PASSED

	`nts` without token.		
Role-Based Restriction	Logged in as PATIENT and tried to fetch `/api/v1/admin/dashboard`.	Request blocked; redirected to login.	PASSED
Session Expiry	Modified token payload to simulate expiry.	User context not set; user prompted to re-login.	PASSED
Logout Verification	Clicked logout button on sidebar.	Tokens cleared, immediate redirect to `auth.html`.	PASSED

7. Testing and Quality Assurance

7.1 Testing Strategy

We focused on manual functional testing rather than automated unit tests. We know that is not ideal for production, but we had to prioritize getting features working. We did write a few integration tests for the billing calculation since that is the most critical part.

Tests we ran:

- Registration with valid data, duplicate username, weak password, missing fields. All blocked properly.
- Login with correct credentials, wrong password, and deactivated account. Also tried SQL injection in the username field to ensure it was safely blocked by parameterized queries.

- Patient CRUD: create, update, search by partial name, filter by blood group, soft delete. Verified deleted patients remain in historical records.
- Room assignment: tried to overbook a room at full capacity. Correctly rejected. Discharged a patient and occupancy decreased.
- Full admission workflow: admit, add treatment records, nurse logs vitals, discharge, generate bill. Verified room occupancy changed at each step.
- Billing: tested same-day discharge (charges 1 day), multi-day stay, missing discharge date (blocks bill generation), discount and tax application, partial payment, refund, and voiding.
- Role access: Nurse accessing /api/v1/admin/** got 403. Doctor accessing doctor endpoints worked.
- Appointment scheduling: created appointment, assigned doctor, verified conflict detection within 30-minute window, marked completed, generated outpatient bill.
- PDF export: generated bill PDF, checked line items and totals. Tested multi-page formatting.
- Dashboard: loaded with 100 records, charts rendered in under 3 seconds.
- Search and pagination: tested combined filters.
- Transport tasks: created a task to move a patient from ICU to General ward, assigned staff, marked complete.
- Vitals logging: nurse recorded vitals for a patient. Verified validation rejected impossible values like temperature above 45 degrees.

- Form validation: client-side blocked empty submissions. Disabled JavaScript and retried. Server-side caught errors.
- Mobile: tested on Chrome DevTools iPhone SE. Tables became scrollable, sidebar collapsed.
- Database constraints: tried deleting a room with active admissions. Foreign key error translated to friendly message.

7.2 Bug Tracking

For the Bug Tracking, each member will pull the project via Github and test it themselves. Whoever found the bug will send it into the group chat, then try to debug it themselves or ask for help.

Room Overbooking: Users could assign patients beyond room capacity. We added the current_occupancy check against capacity in the service layer, wrapped in @Transactional.

Duplicate Bills: Double clicking "Generate Bill" created two bills. We added a unique constraint on bills.admission_id and caught DataIntegrityViolationException.

Appointment Conflicts: Initially doctors could be double booked. We added the 30-minute window conflict check in AppointmentService.

Lazy Loading Errors: When fetching treatment records for the PDF, Jackson tried to publish lazy-loaded associations outside the session, causing errors. We fixed this with @JsonIgnoreProperties annotations on entity relationships.

7.3 Known Limitations

- No comprehensive unit test suite. Only manual testing and a few integration tests.
- No automated database backups configured yet. Railway may have them, but we have not verified.
- No real-time updates. The dashboard and lists require manual refresh.
- English only. No localization.
- No offline support. Internet is required.
- Bills link to either admissions or appointments but not both. We enforce this in code but not with a database check constraint.
- Email notifications were planned but not implemented due to time constraints.

8. Deployment

8.1 Deployment Process

We deployed on Railway. The steps:

- Signed up at https://railway.com/ with GitHub.
- Created a new project and added a MySQL database from Railway's template.
- Railway provided connection credentials (host, port, username, password). We stored these in environment variables, never in the code.
- Connected our GitHub repository to Railway. Railway auto detected the Spring Boot application from pom.xml.

- Set environment variables in Railway's dashboard: DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET.
- Railway built the Maven project and deployed it. We checked the build logs for issues.
- Ran schema.sql and seed-clean.sql against the Railway MySQL instance using the MySQL CLI.
- Used the default railway subdomain.
- Verified all core features on the deployed URL.

9. User Guide

9.1 Getting Started

Once we deploy the application the user can access it at https://codestrike-hospitalmanagement.up.railway.app/auth.html. When they open the root URL it redirects to auth.html for login. The admin creates staff accounts and patients can register themselves.

Some test accounts:

- Admin: admin / pass
- Receptionist: receptionist / pass
- Doctor: dr_an / pass
- Nurse: nurse_mai / pass
- Ward Boy: wardboy_hoa / pass
- Patient: patient_lan / pass

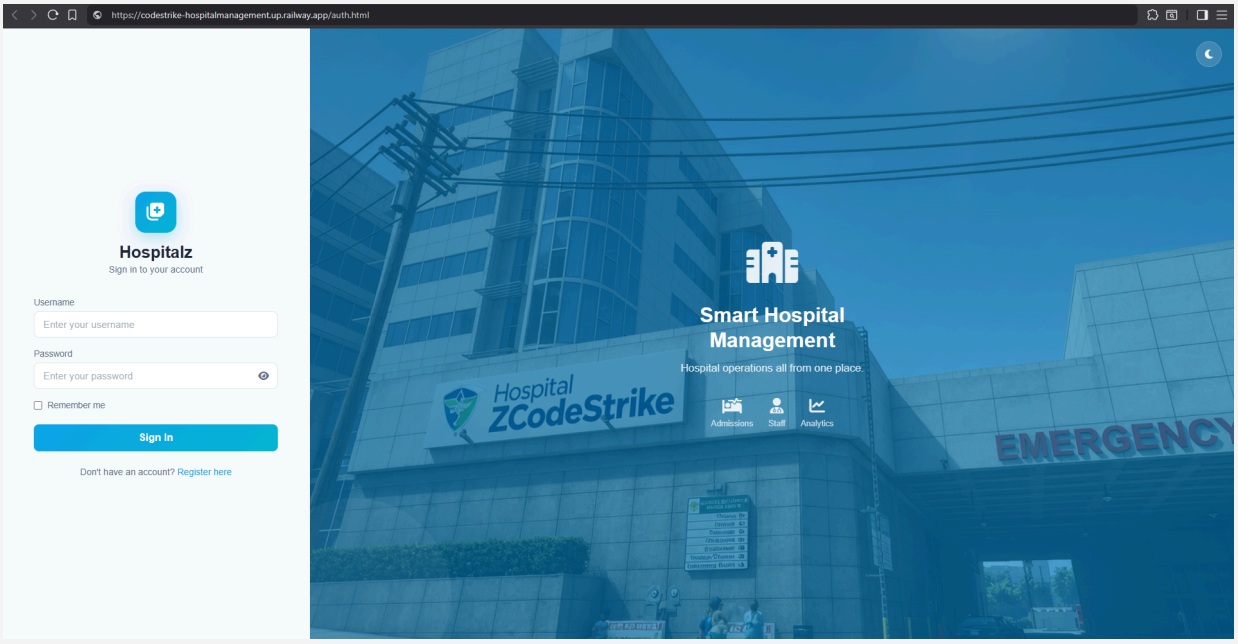

Figure 9.1.1: Login Page

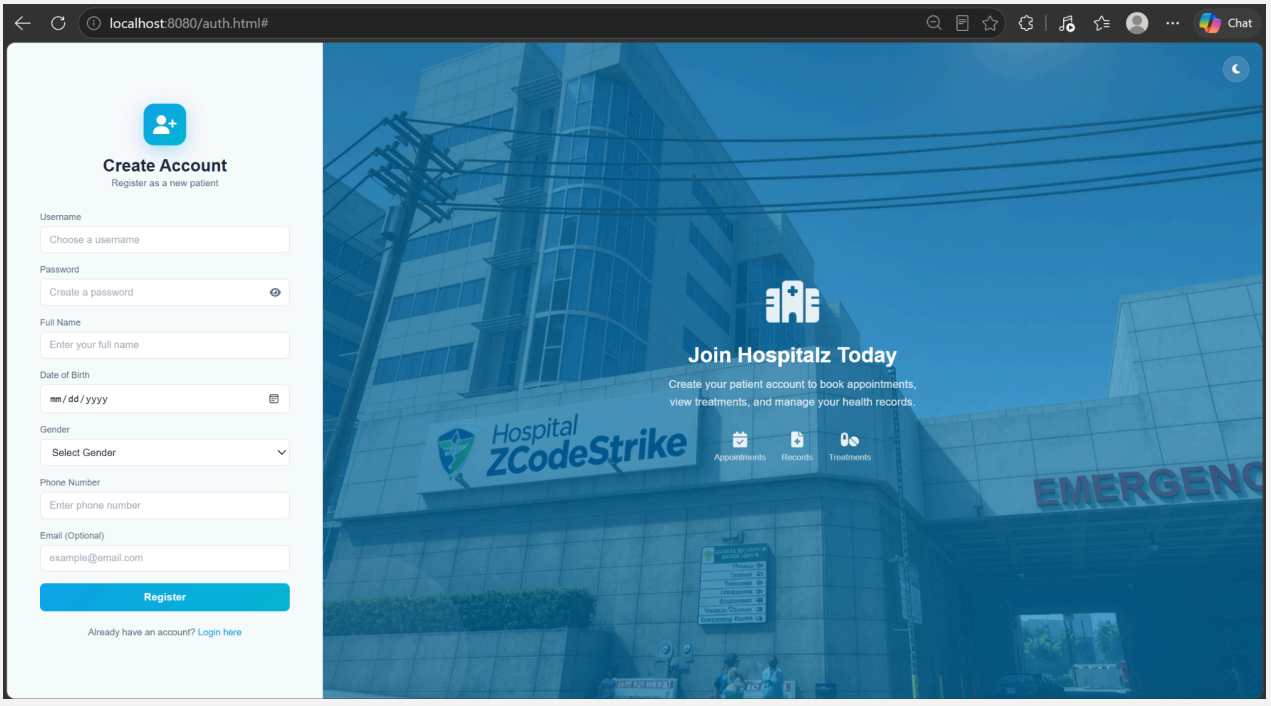

Figure 9.1.2: Register Page

9.2 System Requirements and Prerequisites

To ensure the application runs smoothly the environment should meet these spec:

- **Operating System:** Windows 10/11, macOS (12.0 or higher), or any major Linux distribution (e.g., Ubuntu LTS).
- **Runtime Environment:** Java Development Kit (JDK) 17 or higher.
- **Database Engine:** MySQL Server 8.0 or newer.
- **Client Web Browser:** Modern web browsers with active JavaScript execution engines, such as Google Chrome, Mozilla Firefox, or Microsoft Edge.

9.3 Environment Configuration and System Launch

Database Instantiation: Open the preferred MySQL relational database management tool (e.g., MySQL Workbench or CLI shell).

- Create a database instance named hospital_db.
- Run the schema.sql script located within the system directory to map out core relational entities (including users, patients, admissions, bills, and rooms).

Externalizing Configuration Settings: Navigate to the core resources directory and access application-local.properties.

- Update the Spring Data JDBC/JPA datasource variables (spring.datasource.url, username, and password) with the localized database credentials.

Compiling and Starting the Application: Open a command terminal inside the project root workspace and run the Maven wrapper command: ./mvnw spring-boot:run.

- Once the log registers a successful initialization, open a browser window and connect to the local server endpoint: http://localhost:8080.

9.4 Initial Enrollment and Authentication Workflows

The platform uses JSON Web Token (JWT) architecture for data security. Access is verified using standard Bearer token handshakes. Below is a simple example diagram:

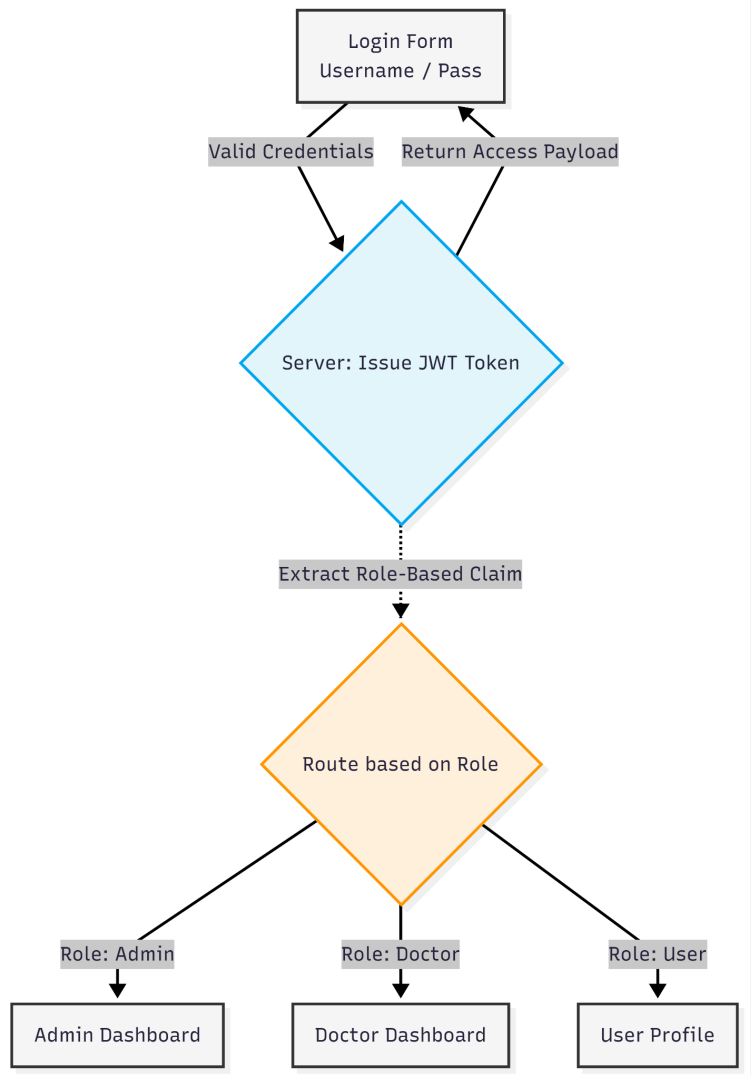

Figure 9.4.1: Workflow

Valid Credentials: Enter login credentials on the Login Form.

Issue JWT Token: If the credentials are correct the system issues a JWT Token with role-based claims.

Access Payload: The token is used to verify access to parts of the application.

- **New Patient Enrollment Routine:**

    - 1. On the welcome landing screen, click the **Register** button.
    - 2. Fill in the fields in the registration form: Username, Password, Full Name, Date of Birth, Gender, Contact Number and Email Address.
    - 3. When the user submits the form the system registers their profile under a ROLE_PATIENT claim. Generates a unique patient identifier with the pattern PAT-YYYY-XXXX (e.g., PAT-2024-0001).
- **Authentication Sequence:**
    - 1. Input user credentials inside the **Login** card.
    - 2. Toggle the Remember Me utility to keep the session active. *(Optional)*
    - 3. If authorization succeeds the system redirects the user to their user dashboard.

9.5 Feature Walkthroughs

The architecture follows a MultiRole Access Control (MRAC) pattern. The following is a deep dive into the transactional workflows categorized by active user roles

9.5.1 Administrative Operations (ROLE_ADMIN)

Administrators have read/write access to all platform data. Here are some of their operations:

- **Executive Dashboard Auditing:** Administrators see a centralized statistical overview, including live bed occupancy metrics, active clinical practitioners, admission ratios and outstanding bill status.
- **Asset and Room Provisioning:** Administrators can add rooms, define room classifications and update operational rates.
- **Clinical Staff Onboarding:** Administrators can onboard clinical staff creating corporate accounts and establishing operational profiles.

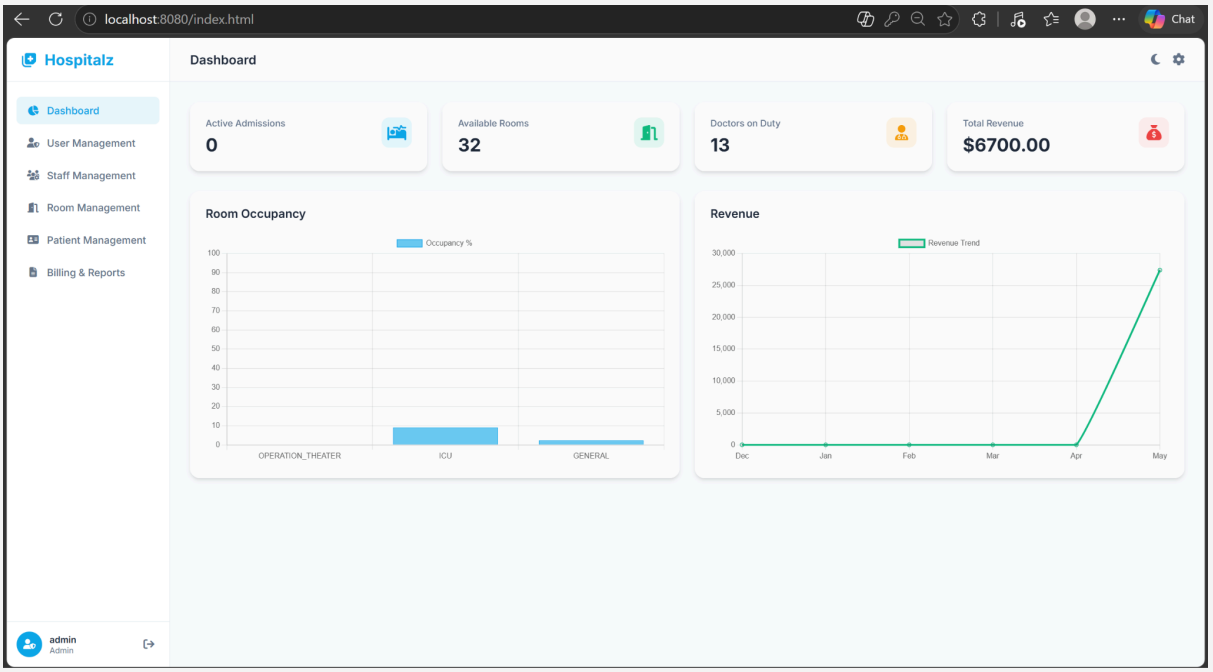

Figure 9.5.1a: Admin Dashboard

9.5.2 Clinical Care Workflows (ROLE_DOCTOR & ROLE_NURSE)

Medical practitioners interact directly with biological metrics, record treatments and issue prescription logs.

Here's an overview of clinical care workflows:

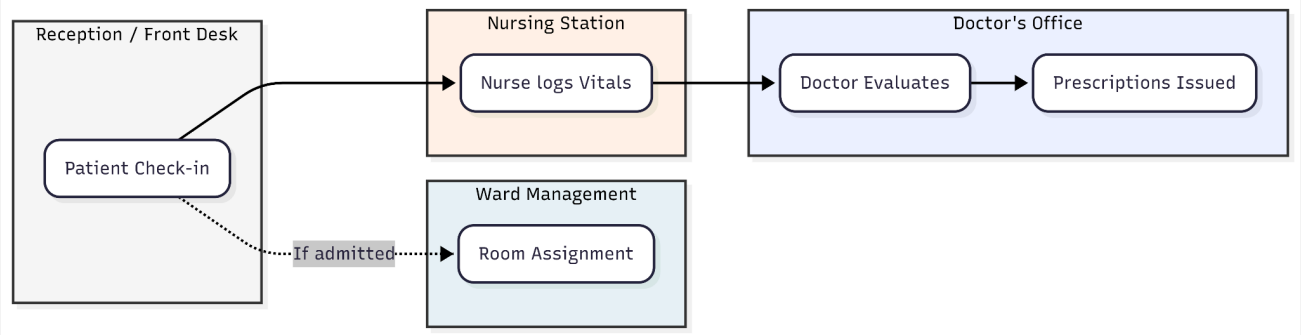

Figure 9.5.2a: Clinic Care Workflow

- **Clinical Cohort Management:** Practitioners access their caseloads through reference tables.
- **Biometric Vitals Logging:**
    - 1. Nurses select any wanted profile and view historical vital signs. Doctors can only select patients assigned to them to view vital signs.
    - 2. The interface loads parameters like heart rate, blood pressure and temperature.
    - 3. New data inputs are posted to VitalsLogService logging the timestamp and operator's digital signature.

- **Prescription Management**: Doctors prescribe treatments by selecting active medical items, specifying quantities and notes, which are then pushed to the patient's record as pending tasks for nurses to administer.

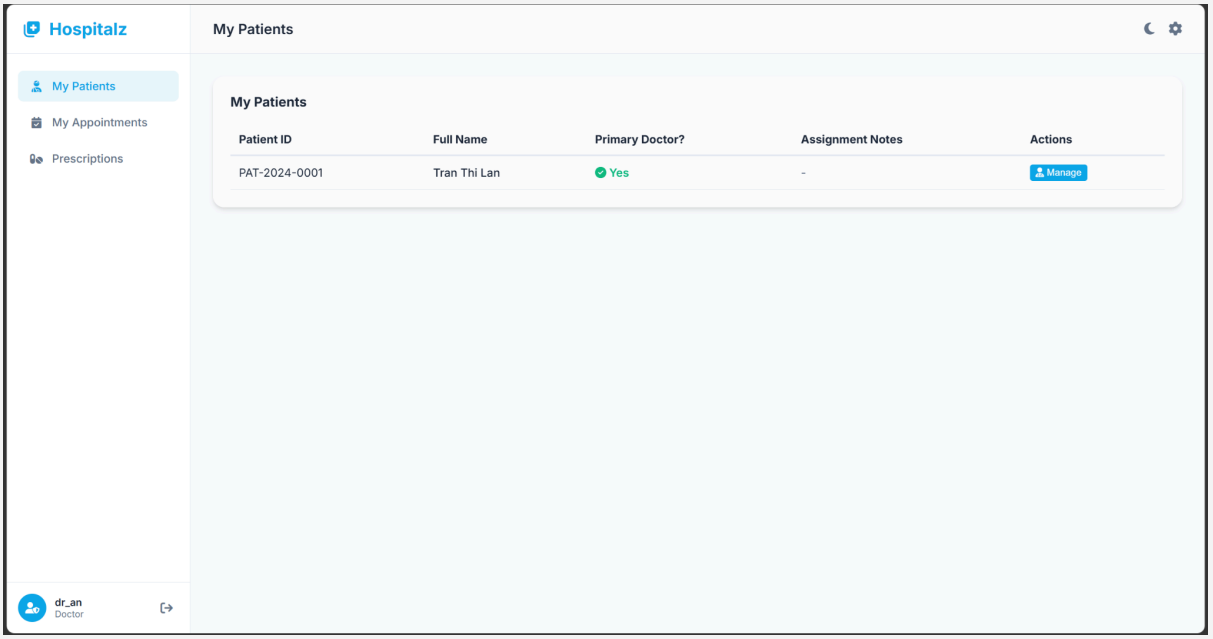

Figure 9.5.2a: Doctor Dashboard

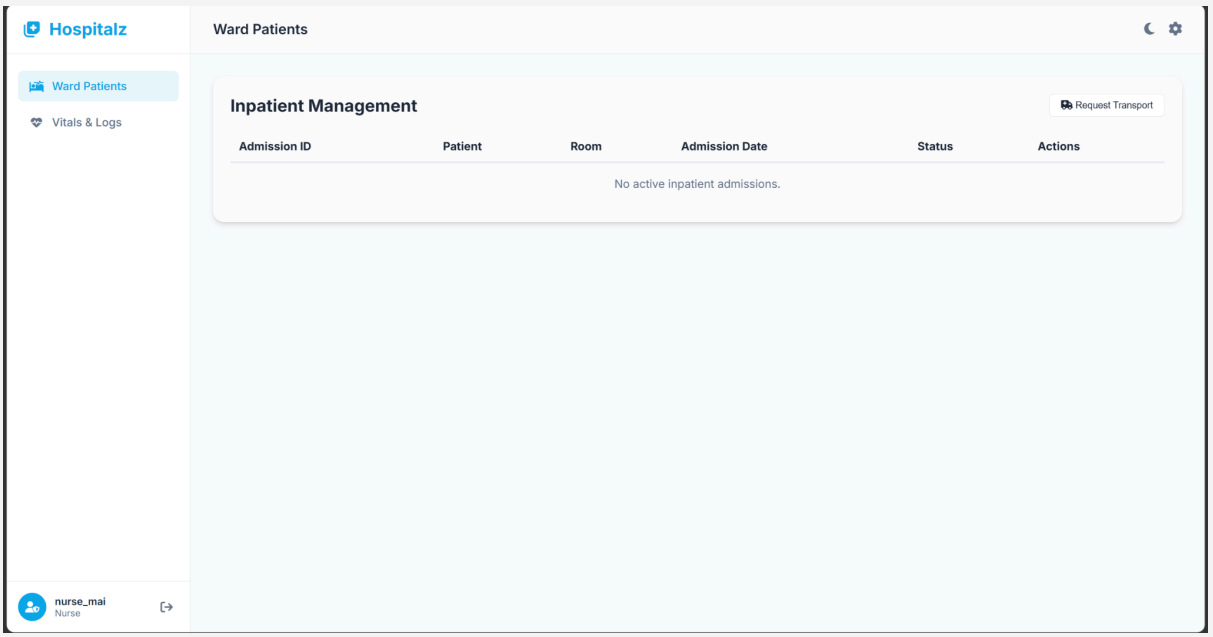

Figure 9.5.2b: Nurse Dashboard

9.5.3 Front-Desk Logistics & Billing (ROLE_RECEPTIONIST)

Reception personnel coordinate system entry points, room occupancy adjustments and final invoice collections.

- **Inpatient Admission Processing:** When a patient requires inpatient monitoring, a clerk processes an AdmissionRequest payload.
- **Financial Auditing and Invoice Settlement:** The front-desk console compiles an itemized financial sheet, calculates mathematical products and processes final payments.

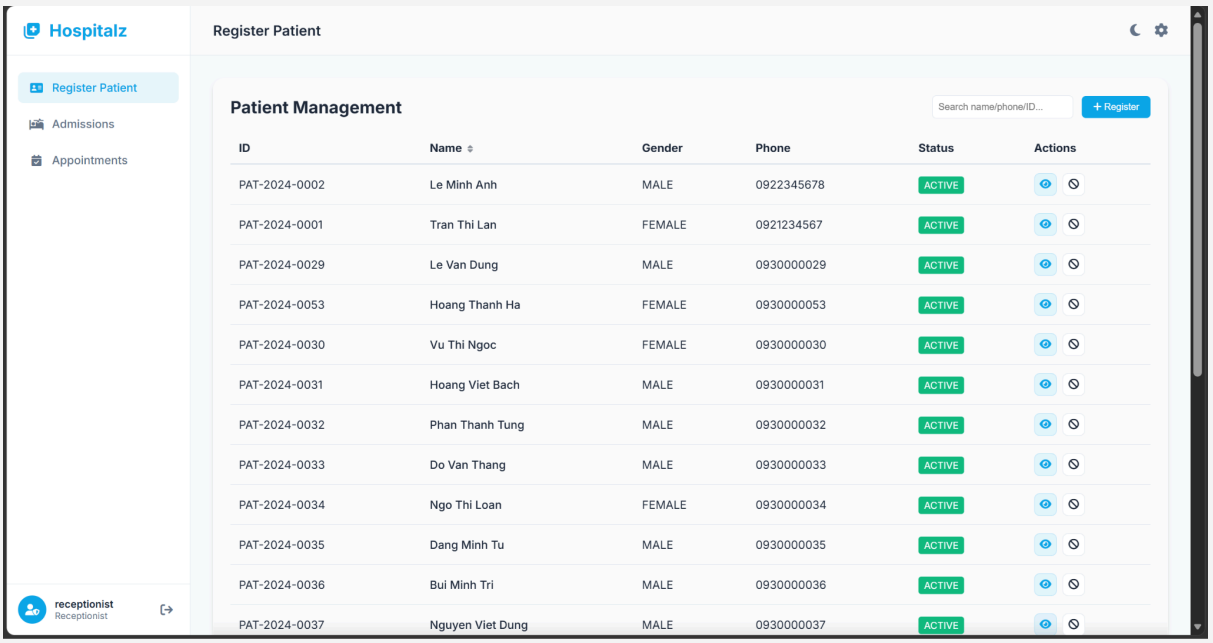

Figure 9.5.3a: Receptionist Dashboard

9.5.4 Patient Self-Service Interfaces (ROLE_PATIENT)

Patients interact with an isolated dashboard that protects private health data.

- **Appointment Management:** Patients request outpatient sessions by selecting an appointment date and specifying their need.
- **Personal Health Ledger & Billing Reviews:** Patients can view their logs and access personal itemized statements. The underlying API ensures the user's logged ID matches the requested data package.

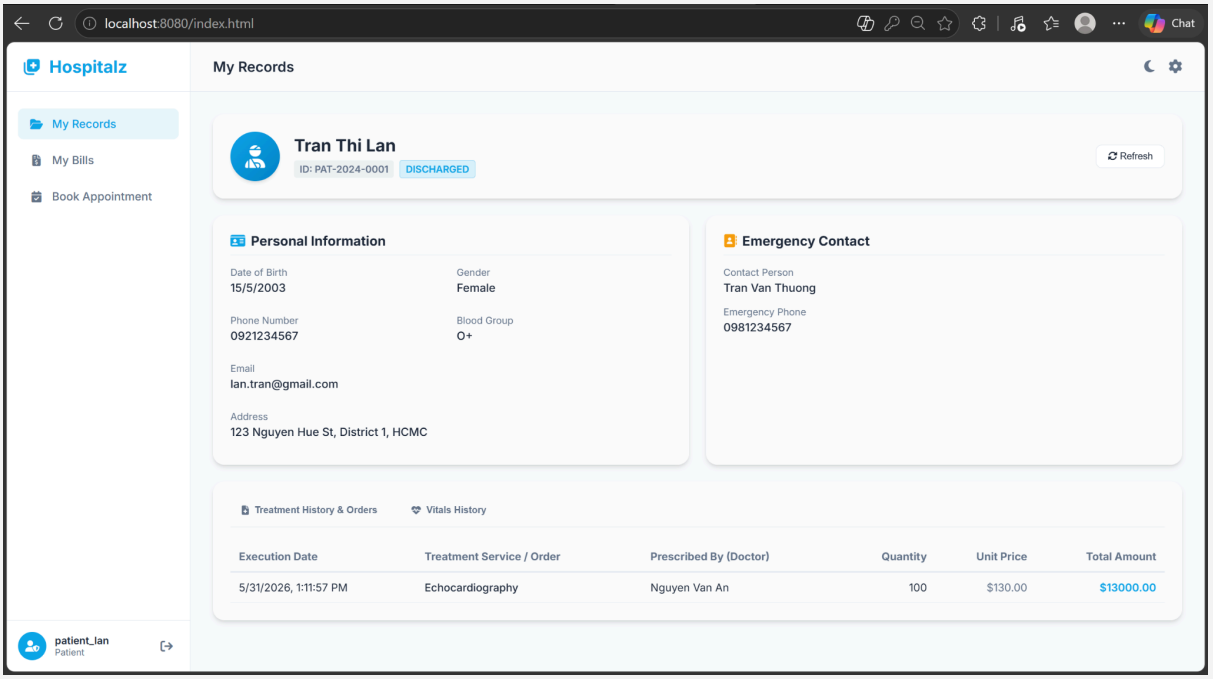

Figure 9.5.4a: Patient Dashboard

10. Team Collaboration

10.1 Team Organization

Our team has 4 members. Though we split work based on strengths, each member still has the right to assist other members if available.

Table 10.1.1: Assigned Task table:

Member	Task
Nguyễn Thế Khoa	Backend: Spring Boot controllers, services, security, billing logic, PDF generation.

Phạm Song Gia Khánh	Frontend: HTML, CSS, JavaScript, dashboard charts, UI consistency.
Nguyễn Hưng	Integration, SQL queries, seed data, documentation, testing coordination, database design.
Trần Đình Hưng	Testing, bug tracking, error handling and validation improvements, UI polish, and deployment setup.

11. AI Log

Tools Used:

Claude: used only during Week 1 for brainstorming and concept validation.

Conceptual Areas Where AI Was Used

11.1 Patient Journey Mapping

What we asked: "Map a patient flow from admission to discharge."

AI suggested: A simple linear flow: Register → See Doctor → Pay → Leave.

What we changed: Designed a non linear workflow with multiple admissions, room transfers, parallel outpatient appointments, transport tasks, and a final discharge bill that aggregates room charges, treatments, and doctor fees.

Reality: AI missed inpatient logistics completely.

11.2 Billing Logic

What we asked: "How should a hospital billing system work?"

AI suggested: A basic per service invoice.

What we changed: Developed an automated billing concept tied to admission duration (daily rate × total days), historical price snapshots for treatments, plus discount/tax handling and partial payment support.

Reality: AI did not understand the need to freeze historical prices or combine inpatient and outpatient charges.

11.3 Soft Delete vs. Hard Delete

What we asked: "Should I use soft delete or hard delete for a hospital system?"

AI suggested: Soft delete is safer for data integrity, but gave generic examples using a simple deleted flag.

What we changed: Implemented it as is_active because medical records require legal preservation. Designed it specifically to protect foreign key relationships in our admission, treatment, and billing history. Ensured

deactivated users cannot log in but their historical records remain intact for audits.

Reality: AI stated the obvious (soft delete is safer). We had to figure out the hospital specific legal and relational reasons ourselves.

11.4 DTOs vs. JPA Entities

What we asked: "Why use DTOs instead of entities in Spring Boot?"

AI suggested: Standard security reasons about mass assignment and decoupling.

What we changed: Applied it specifically to our multi role system. We created RegisterRequest, AdmissionRequest, and BillRequest DTOs because we needed to prevent users from injecting fields like role or id during registration. AI did not mention our specific vulnerability where a Patient could self-assign as ADMIN.

Reality: AI gave textbook definitions. We mapped the concept to our actual API security requirements.

11.5 SPA Architecture Communication

What we asked: "How does a vanilla JavaScript SPA talk to Spring Boot?"

AI suggested: Use REST APIs and CORS. Gave a generic diagram of a frontend calling /api.

What we changed: Designed our own static resource structure (auth.html, index.html served from /static). Built a custom api.js wrapper around Fetch API to handle JWT Bearer tokens and 403 redirects. AI did not mention stateless JWT storage in sessionStorage/localStorage.

Reality: AI confirmed the general approach was valid. The actual implementation details were our own.

11.6 Database Normalization

What we asked: "Explain 3NF and give a hospital database example."

AI suggested: A generic example separating "doctor phone number" from an appointments table.

What we changed: Applied it to our actual schema. For example, we separated doctor profiles, room details, and treatment catalogs into their own tables instead of duplicating them across admissions and bills. We also made an intentional exception for billing snapshots (denormalization) to preserve historical prices.

Reality: AI gave a classroom example. We had to analyze our own functional dependencies and decide where to intentionally break normalization for billing integrity.

12. Conclusion and Reflection

12.1 Project Summary

We built a Hospital Management System covering patient records, room management, doctor assignments, appointments, treatment records, vitals logging, transport tasks, and automated billing. It has 12 database tables, 6 user roles with access control, and 3 features: dashboard charts, automated PDF billing with payment tracking, and appointment conflict detection. The system

stack is Spring Boot 4.0.6, Java 21, MySQL, vanilla JavaScript SPA, and deployment on Railway.

We achieved most of our initial goals. The appointment and transport task features were more complex than expected but we got them working. We had to drop email notifications due to time constraints.

12.2 Lessons Learned

Technical:

- Spring Boot is powerful but takes time to learn. Week 1 was mostly understanding dependency injection and JPA. After that, development accelerated.

- Database design should be finalized early. Schema changes cascade through entities, DTOs, services, and views. We should have spent more time on the ER diagram in Week 1.
- BigDecimal is mandatory for money. We learned this the hard way in a previous lab with double rounding errors.
- Server-side validation is essential. Client-side JavaScript can be bypassed.
- JWT authentication fits SPAs well, but token storage in localStorage has security trade-offs. For a production system we would consider httpOnly cookies.

Project Management:

- Weekly milestones are necessary. In Week 2 we over-polished the UI and fell behind on billing logic, causing a rush in Week 4.
- A shared API contract document prevents integration headaches.
- Git feature branches let us work in parallel safely.

Teamwork:

- Communication beats individual coding speed. Ten minutes explaining a plan saves hours of merge conflicts.
- Pair programming helped on complex parts like billing calculation. We caught edge cases together that one person might have missed.

12.3 Future Enhancements

1. Medicine inventory management with stock alerts and supplier integration.
2. Feedback system for users
3. Better security against cyper-attack.
4. Insurance claim processing and co-pay calculations.
5. Lab test orders and result uploads.
6. Mobile app for staff.
7. Real-time WebSocket notifications.
8. Multi-tenancy for multiple hospital branches.
9. AI diagnosis suggestions.
10. Email and SMS notifications for appointments and billing

13. References

Spring Boot: Best practices for scalable applications. (2025, October 29). *SpringBoot: Best Practices for Scalable Applications*. https://www.codewalnut.com/insights/spring-boot-best-practices-for-scalable-applications

Hai, V. D. (2011). A web-based electronic medical records and hospital information system for developing countries. *ResearchGate*. https://www.researchgate.net/publication/239524799_A_web-based_electronic_medical_records_and_hospital_information_system_for_developing_countries

Singh, K., & Singh, K. (2025, April 7). *Complete guide to JWT and how it works*. Security Boulevard. https://securityboulevard.com/2025/04/complete-guide-to-jwt-and-how-it-works/

Railway. (2026, May 29). *Quick Start tutorial | Railway Docs*. Quick Start Tutorial | Railway Docs. https://docs.railway.com/quick-start?fbclid=IwY2xjawSJDD5leHRuA2FlbQIxMABicmlkETFLMnUwRFcwdkQzTm9JUUw0c3J0YwZhcHBfaWQQMjIyMDM5MTc4ODIwMDg5MgABHoNcQ8SVmyAoipB-zxLvLQZo8Ar9K8mmUaeSTuGwYMHuNeIMyQyCsGZCzkK5_aem_8Yaf7noyYT2duJ5GTHkUpQ

Interior Health. (n.d.). *AH3000 – Assignment of hospital rooms to support patient privacy, dignity and safety* [PDF]. Interior Health. https://www.interiorhealth.ca/sites/default/files/policies/admin/AH%20-%20Patient_Client%20-%20%20Relations_Care/Assignment%20of%20Hospital%20Rooms.pdf